

Smart vehicle Inspection and Repair Estimator



By

Rawan Osama Abdelalim Mousa 224159

Ranya Sherif Salah Eldin Elleithy 226701

Mohamed Ayman El Sayed Khedr 219203

Ziad Khaled Eid Abdelsalam 208605

Ramez Ashraf Abdulmoniem Mahmoud Nasr 225652

A Dissertation submitted to the Faculty of Engineering, The British University in Egypt, in
Partial Fulfillment of the requirements for the bachelor's degree in electrical engineering

Supervisor(s): Dr. Sally Saad

June, 2026

Electrical Engineering Department

Computer Engineering Programme

Smart vehicle Inspection and Repair Estimator

Dissertation Approved:

Dr. Sally Saad

Committee member

Committee member

External committee member

Affiliation:

Dean Faculty of Engineering

Acknowledgments

We wish to extend our sincere appreciation to our mentor, Dr. Sally Saad, for all her constructive suggestions, assistance, and help in making this final graduation project possible and successful. Her professionalism and dedication were vital in determining the focus of our study as well as the standard of work. During the entire duration of our project, she always motivated us to analyze things and solve problems using a critical mind and with utmost professionalism.

We would like to thank the Faculty of Engineering at The British University in Egypt for offering us an environment that facilitates learning and has helped us complete this project successfully, owing to its high standards of education and learning.

Abstract

Typically, vehicle damage inspection is done manually by inspectors or repair specialists, a process that can be time-consuming, subjective and inconsistent, particularly in insurance and repair-cost estimation workflows. The purpose of this project is to develop a smart vehicle inspection and repair estimator system based on deep learning and computer vision technologies that automatically recognize external vehicle damage visible in images uploaded from the front and/or rear of the vehicle. YOLOv11m-seg is the instance segmentation model used in this study which is trained on Car Damage Detection Dataset (CarDD) comprising six damage categories: dent, scratch, crack, glass shatter, broken lamp, and flat tire. The model identifies and localizes damaged areas via bounding boxes and segmentation masks. The classified damage is then sent to a rule-based damage severity estimation module to identify minor, moderate and severe damage. There is also an integrated Repair cost estimation module that offers an approximate repair price range depending on the responses gathered from the survey and the pricing data obtained from the workshop. The final system is integrated with an app for mobile that enables the user to upload a picture of a vehicle, get damage detection results, severity and cost estimates, download a report, use a chatbot and give feedback. The experimental results demonstrated that the proposed YOLOv11m-seg model showed promising performance in terms of overall detection and segmentation with the approximate box mAP@0.5 and mask mAP@0.5 performance of 0.769 and 0.763 on the held out test set respectively. The system was best on image distinctive damage like glass shatter, flat tire and broken lamp, whilst smaller surface damages like cracks, scratches and dents were more difficult. In general, the following system offers a practical initial assessment tool that can be used to assist vehicle owners, repair shops, and insurance related processes, but the given repair cost should be considered as an approximation and not as a straight-up quote.

Table of Contents

Acknowledgments	ii
Abstract.....	iii
List of Figures.....	viii
List of Tables	ix
List of abbreviations	x
Chapter 1: Introduction.....	1
1.1 Background.....	1
1.2 Problem statement	1
1.3 Research Objectives	2
1.4 Research Questions	2
1.5 Research methodology	2
1.6 Project Scope	3
1.7 Project Limitations	3
1.8 Report Organization	4
Chapter 2: Background.....	5
2.1 Artificial Intelligence and Deep Learning in Vehicle Inspection.....	5
2.2 Convolutional Neural Networks (CNNs) for Image-Based Inspection	5
2.3 Current Vehicle Inspection Practices and the Need for Smart Systems.....	6
2.4 Advantages and Limitations of AI-Based Vehicle Inspection	7
2.5 Computer Vision Tasks in Vehicle Damage Assessment.....	7
2.5.1 Image Classification	8
2.5.2 Object Detection.....	8
2.5.3 Instance Segmentation.....	9
2.6 Object Detection vs Segmentation for Damage Analysis.....	9

2.7	YOLO-Based Models	10
2.8	YOLOv11m-seg Architecture	10
2.9	Vehicle Damage Types and Visual Challenges	12
2.10	Severity Estimation in Vehicle Damage Assessment	13
2.11	Repair Cost Estimation in Vehicle Inspection Systems	13
2.12	Mobile Application-Based Inspection Systems	14
Chapter 3: Literature Review		15
3.1	Related Work Overview	15
3.2	Summary and Research Gap Analysis	19
Chapter 4: Methodology		20
4.1	System Overview	20
4.2	Dataset Description	21
4.2.1	Car Damage Detection Dataset	21
4.2.2	Repair Price Dataset	21
4.2.3	FixZone Chatbot Dataset	22
4.3	Data Preparation and Pre-processing	23
4.4	Model Selection and Justification	23
4.5	Model Architecture and Training Configuration	24
4.5.1	Training Configuration	24
4.5.2	Model Workflow	25
4.6	Evaluation Metrics	26
4.7	Post-Processing and Confidence Threshold Selection	27
4.8	Severity Estimation Methods	28
4.9	Repair Cost Estimation Method	30
4.10	Chapter Summary	31

Chapter 5: Results and Comparisons	32
5.1 Implementation Environment	32
5.2 Dataset Split and Class Distribution	32
5.3 Training Performance and Loss Curves.....	33
5.4 Validation Results	34
5.5 Testing Results.....	35
5.6 Confusion Matrix Analysis	36
5.7 Class-wise Performance Analysis.....	38
5.8 Test Results Analysis	39
5.9 Post-Processing Results.....	41
5.10 Severity and Cost Estimation Results.....	42
5.11 Discussion.....	43
5.12 Chapter Summary	44
Chapter 6: Mobile Application Development and Features.....	45
6.1 Overview of the Mobile Application	45
6.2 Application Development Tools and Technologies	45
6.3 System Workflow	46
6.4 User Interface Design	47
6.5 Main Application Features.....	48
6.5.1 Login and Signup	48
6.5.2 Uploading and Scanning Phase	49
6.5.3 Damage Analysis Result Page	49
6.5.4 Recommended Repair Shops.....	50
6.5.5 Generated Vehicle Damage Assessment Report	51
6.5.6 Chatbot Program.....	52

6.5.7	Account, Settings, and User Information	53
6.5.8	Feedback and Rating	54
6.6	Firebase Storage and User Data Handling.....	55
6.7	Advantages of the Mobile Application.....	55
6.8	Application Limitations.....	55
6.9	Chapter Summary	56
Chapter 7: Conclusion and Future work.....		57
7.1	Conclusion.....	57
7.2	Limitations.....	58
7.3	Future work	58
References		59

List of Figures

Figure 1: Example of deep learning-based feature extraction using CNN.....	6
Figure 2: Different types of detecting vehicle using artificial intelligence	8
Figure 3: The original architecture of the YOLO object detection framework.....	10
Figure 4: YOLOv11 architecture with backbone, neck, and head components].....	11
Figure 5: Examples of common vehicle damage types used in image-based inspection.....	12
Figure 6: System Architecture of the Proposed Vehicle Damage Detection Application	20
Figure 7: Distribution of approximate vehicle repair costs reported by survey respondents	22
Figure 8: Training loss curves and performance metrics of the proposed YOLOv11m-seg model	34
Figure 9: Validation performance of YOLOv11m-seg on the CarDD validation set	35
Figure 10: Test performance of YOLOv11m-seg on the CarDD test set	35
Figure 11: Test performance metrics of the YOLOv11m-seg model	36
Figure 12: Test confusion matrix of the proposed YOLOv11m-seg model	38
Figure 13: Real-World Vehicle Damage Detection and Localization Result	40
Figure 14: Example of damage detection with moderate severity, and estimated repair cost.....	43
Figure 15: General workflow of the FixZone mobile application system	46
Figure 16: The login, home screen and damage analysis pages.....	48
Figure 17: Repair Shops Page	50
Figure 18: Generated report Layout.....	51
Figure 19: FixZone rule-based chatbot interface	52
Figure 20: Settings screen of the FixZone application.....	53
Figure 21: Feedback screen of the FixZone application	54

List of Tables

Table 1: Comparison of prior studies on vehicle damage and cost estimation	17
Table 2: YOLOv11m-seg training configuration	25
Table 3: Evaluation metrics used for model assessment	27
Table 4: Class-specific confidence thresholds.....	28
Table 5: Class-specific severity thresholds	29
Table 6: Approximate repair cost ranges used by the system.....	31
Table 7: Class distribution across the CarDD dataset splits.....	33
Table 8: Summary of post-processing effect on test predictions.....	41
Table 9: Examples of severity and cost estimation outputs.....	42

List of abbreviations

- AI: Artificial Intelligence
- AUC: Area Under the Curve
- CarDD: Car Damage Detection Dataset
- CSV: Comma-Separated Values
- CNN: Convolutional Neural Network
- DL: Deep Learning
- IoU: Intersection over Union
- mAP: mean Average Precision
- ML: Machine Learning
- Mask R-CNN: Mask Region-Based Convolutional Neural Network
- NMS: Non-Maximum Suppression
- PDF: Portable Document Format
- SSD: Single Shot Detectors
- YOLO: You Only Look Once
- YOLOv11m-seg: YOLO Version 11 Medium Segmentation Model

Chapter 1: Introduction

1.1 Background

Due to the increasing number of vehicles worldwide, the rate of road traffic accidents and insurance claims has also grown. The expansion imposes a lot of demand on the insurance firms and the auto repair shops, which must be able to estimate the damage fast, precisely and equitably. The conventional method used to assess vehicle damage is mostly based on the use of human experts that carry out the necessary manual inspections, which is a time-consuming process and too subjective of an evaluation [1].

The methods of manual inspection are usually subjective, inaccurate, and slow in the processing of the claims. Moreover, the use of human insurers adds operation expenses and puts the insurance business at risks of fraud (damages exaggeration or misclassifying the conditions of vehicles). These difficulties influence customer satisfaction and distrust in the insurance services adversely [2].

The recent developments in Artificial Intelligence (AI), especially in the computer vision and deep learning domains, have provided new opportunities of automating the process of vehicle damage assessment. Through vehicle image analysis, AI-based systems can detect, localize and categorize damage types in a rapid and uniform way. This type of automation enhances efficiency, minimizes fraud, and increases transparency in the insurance claim process.

1.2 Problem statement

Alongside the growth of the digital technologies, the vehicle damage evaluation is still predominantly manual, which is ineffective and inappropriate to manage the ever-growing number of transportation accidents. The processes of manual inspection would lead to slothful claim procedures and irregular assessment of the damage. The absence of a common way of assessment results in uneven repair price estimations, which often results in conflicts between the insurers, repair shops and their owners. Moreover, manual inspection is subjective and, therefore, leaves the system vulnerable to fraudulent activities causing financial losses and lack of trust of the customers. These issues raise the need to have an automated, error-free, and scalable solution to assess that of

vehicle damage. So, the problem in this project is not only to identify whether a vehicle is damaged, but also the damaged area, the level of the damage and finally to relate the damage to an approximate repair cost, which will be available to the user through an application.

1.3 Research Objectives

The main idea of the project is to design and develop an integrated AI-based system to solve the above-mentioned problems. The primary objectives are:

- To create an artificial intelligence system that will be able to detect and locate vehicle damage in images automatically.
- To categorize the damage identified as per its severity levels to categorize the minor, moderate and severe damage.
- To estimate costs of repairs by associating their observed characteristics of damage.
- To develop a convenient application that allows quick assessment of the damage of vehicles and claim settlement.

1.4 Research Questions

Our research focuses on four main questions that we need to solve to make this system work:

- What is the performance of YOLOv11m-seg on detecting and segmenting the visible external damage with real-world vehicle images?
- Which types of damage are identified best, and which types of damage are still tricky?
- What are the methods to assess damage severity in a conservative way by using the concept of segmentation mask area and damage type?
- Is it possible to develop a mobile application that incorporates the proposed detection process, the proposed severity estimation process and the cost estimation process, facilitating the reporting process?

1.5 Research methodology

To construct the suggested approach that can provide an AI-powered vehicle damage recognition, an experimental research strategy is applied in this paper. The main data source for training the proposed AI-based model will be the Car Damage Detection Dataset (CarDD). It

provides annotated examples of vehicle damage that are useful for training and validating the suggested vehicle damage segmentation model.

To perform the vehicle damage detection and segmentation task, the suggested method uses the architecture of YOLO11m-seg. This model was chosen because it performs effectively when it comes to real-time object detection, segmentation, and localization of the detected objects. Thus, the model can identify and localize fine-grained damages. In the current experiment, it will be used to detect and identify the types of vehicle damage.

The evaluation procedure involves using standard object detection and segmentation performance metrics such as precision, recall, and mean Average Precision (mAP). These criteria allow assessing the effectiveness of the suggested model in recognizing vehicle damages and localizing their positions on the image.

1.6 Project Scope

The scope of this project focuses on the detection and analysis of visible external vehicle damage using image-based deep learning techniques. The system will analyze the images of the vehicles uploaded by the user and determine typical damage categories such as dents, scratches, cracks, glass breakage, broken lamps and flat tires. The project also includes a module to estimate the severity of the damage according to the type of damage and mask area ratio, and a repair cost estimation module based on the cost ranges collected in the surveyed responses and based on the information gathered in the workshops.

Besides the artificial intelligence model, the project features the design and development of a mobile application interface where the user can upload an image, see the damage shown, view the damage level estimate, receive a proposed repair price, download a damage report, talk to a rule-based chatbot and provide feedback. The project does not aim to replace professional vehicle inspection completely, but rather to provide an initial automated assessment that can support vehicle owners, repair shops, and insurance-related workflows.

1.7 Project Limitations

Although the proposed system provides a practical approach for automated vehicle damage inspection, it has several limitations. Firstly, it relies on the quality of the image that is uploaded. Poor ambient light, blurring, reflections, unusual camera angles or partial visibility of the damaged

area can affect detection accuracy. Furthermore, there are some types of damage that are hard to observe visually, such as scratches, dents and cracks, which may be small or thin.

The other drawback is that the severity is computed according to the percentage of area covered by the mask compared to the total image area, not the actual size of the vehicle part. The severity logic must be conservative as there is no car-part segmentation model and to prevent overestimation of damage. Furthermore, the estimation of the repair cost is only an approximation, based on rules, because actual repair prices vary by vehicle brand, availability of spare parts, workshop prices, labor prices, and changes in the market. Thus, the cost result must be considered as an estimate of the range of costs, and not as an exact repair quotation.

1.8 Report Organization

This report has been divided into six chapters. The project background, problem statement, project aim, project objectives, design questions, project scope, project limitations, and the structure of the report are introduced in Chapter 1. Theoretical background is provided in Chapter 2 to understand the project, which includes the background of artificial intelligence, deep learning, convolutional neural networks, computer vision tasks, YOLO based models, instance segmentation, vehicle damage types, severity estimation, repair cost estimation, and mobile application-based inspection systems.

Related studies on automated vehicle damage detection, damage detection using the YOLO algorithm, segmentation-based damage analysis, damage severity estimation, damage cost estimation, and user-oriented vehicle inspection system are reviewed in Chapter 3. It also outlines the research gap and the value of the proposed research project. The methodology and design of the system are explained in chapter 4, which includes the proposed workflow of the system, dataset selection, CarDD dataset description, data preprocessing, model selection, YOLOv11m-seg training methodology, severity estimation method, repair cost estimation method, requirements of mobile application, application design, features, and system architecture.

The implementation, experiments, results and discussion are presented in Chapter 5. It contains the environment in which it was implemented, software tools used, end-to-end experiment with YOLOv11m-seg, evaluation metrics, training results, validation results, testing results, segmentation output, severity and cost estimation results, end-to-end testing, discussion, and

implementation challenges. Lastly, Chapter 7 presents a summary of the key findings and future research.

Chapter 2: Background

2.1 Artificial Intelligence and Deep Learning in Vehicle Inspection

Artificial Intelligence (AI) is described as the ability to make computer systems perform activities that mainly require human intelligence and complex tasks. Computer vision, a subfield of artificial intelligence (AI) is a crucial field that deals with complex tasks such as image classification, object detection, and image segmentation [3]. In the past few years, the use of AI-based systems has rapidly increased due to the advancements of computational powers and availability of large datasets in multiple fields, making it possible to use such systems in industrial environments. Within industrial environments, AI-based systems have been implemented in the automotive industry, such systems help manufacturers to determine the presence of various forms of car defects like scratches or dents, by using multiple deep learning approaches therefore reducing the work required from employees in the inspection process as well as increasing the efficiency of detecting any form of defects [4]. As a result, AI-based vehicle inspection systems provide a hopeful substitute for traditional inspection techniques.

2.2 Convolutional Neural Networks (CNNs) for Image-Based Inspection

Deep learning is very effective in assignments such as image classification, object recognition and image analysis, particularly the convolutional neural networks (CNN) [3]. Unlike the conventional techniques, which extract basic features, such as edges of the object, CNNs are capable of identifying more complex objects and hierarchical features present in data. CNNs can effectively extract representations of imagery data and provide high-dimensional data into meaningful representations, as they can formulate spatial hierarchies of data using convolutional and pooling layers. As an example, the early layers can learn the lower-level pattern of edges on the other hand, the subsequent layers combine the earlier pattern and information to discover the higher-level objects and forms [5]. The CNNs are highly applicable when it comes to identifying automotive damage as a large collection of automobile images can be trained and can therefore be able to detect the various types of vehicle damage such as scratches, dents, cracks among others. CNNs enhance the speed and quality of damage detecting algorithms by creating information at

various levels of abstraction, which makes them a potent solution to this task [5]. Figure 1 shows how CNN extracts high level features from images using multiple convolution and pooling layers.

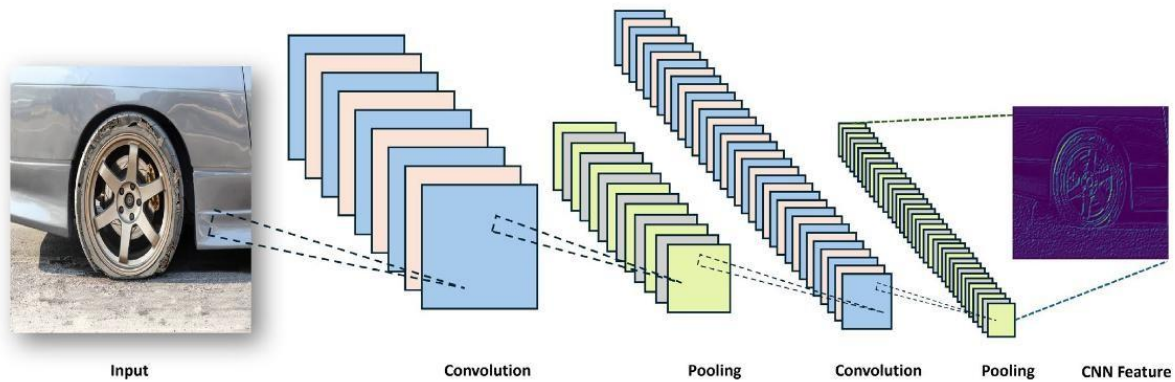


Figure 1: Example of deep learning-based feature extraction using CNN [3]

2.3 Current Vehicle Inspection Practices and the Need for Smart Systems

The existing car inspection system is mostly based on a manual examination of the car, done by a mechanic, or one with experience in the field. Here, the exterior of the car is analyzed and examined to identify the parts of the car that had been damaged, the extent to which it was damaged, and then the total approximate cost of repair or cost of the spare parts is calculated depending on the information obtained. The insurance inspection in contrast to the inspection of the mechanic entails additional steps like documentation, verification and approval of claims which increases the time further taken to complete the evaluation. Even though the manual inspection types are highly widespread nowadays, they are time-consuming, subject to human error and inconsistency, and are not very reliable when applied to large-scale operations, which is why the necessity of the smart systems is extremely high [4], [6]. Furthermore, vehicle owners must visit workshops or insurance inspection centers which can be far away from them, even the inspectors sometimes need to visit the vehicle owner's location for inspection, making the process inconvenient and time-consuming either way [5]. These drawbacks highlight the necessity of a smart vehicle inspection system that can use image analysis to automatically evaluate damage. Smart inspection systems use artificial intelligence and deep learning methods to be used as a support system to inspectors to help both them and the users by providing faster, more consistent, and easily accessible damage assessments [7].

2.4 Advantages and Limitations of AI-Based Vehicle Inspection

Smart vehicle inspection systems provide many benefits over manual inspection techniques. Automated systems significantly decrease the inspection process time and bias in assessment, as the manual process is subjective. These systems also improve accessibility by allowing inspections to be conducted remotely by uploading car images into the system or application, anywhere and at any time, thereby reducing the need for in-person visits to inspection centers [7].

Apart from insurance and repair processes, smart inspection systems may be effective to sellers of second-hand vehicles or internet sites. The system will help sellers to be able to make initial assessment of the damage on their vehicles online, remotely and at any time, which will enable them to be able to know the correct price at which to sell their vehicles and improve the entire selling process. This is an advantage for both the dealers and acquirers, as it helps dealers to save inspection processes, and on the other hand, buyers can have clear information on the condition of the cars. Business wise, this kind of system minimizes operations, enhances customer contact and have better risk assessment [5].

Nevertheless, these systems have certain limitations even with these advantages. The quality of their performance depends on the availability of large and diverse datasets and image quality may result in errors. Moreover, acquiring trustworthy repair cost information is especially difficult, because such data is mainly regarded as private data by service providers and it frequently changes due to the market conditions. Therefore, obtaining precise repair costs is one of the toughest challenges in system development. Moreover, other major challenges happen during implementation, such as optimizing models and addressing explainability issues that must be addressed to ensure reliable and fair deployment [7].

2.5 Computer Vision Tasks in Vehicle Damage Assessment

Computer vision is one of the most crucial elements in the automated vehicle damage assessment system as it enables a system to read and understand the information in vehicle images. In this project, we adopted computer vision to detect damaged regions, classify damage types and generate segmentation masks that indicate damaged regions. The primary computer vision-related tasks for this system are image classification, object detection and instance segmentation.

2.5.1 Image Classification

Image classification is the task of labelling the whole image with one label. In vehicle damage assessment, a classification model may determine whether an image contains damage or no damage or classify the general type of damage present in the image. But this classification is not sufficient for this project, as it does not demonstrate the site of damage and its size. For instance, it can determine that a picture has a scratch, but it cannot tell where it is and how big it is.

2.5.2 Object Detection

Object detection is one of the main computer vision tasks that demands recognizing and locating objects in an image by using bounding boxes. In contrast to image classification, which gives the complete image one label only, object detection allows the identification and localization of a specific area or object within the image, making it especially effective for analyzing vehicle damage, as damage in vehicles is mainly limited to specific parts like doors, bumpers, fenders, or headlights, which require accurate localization instead of overall image labeling [5].

Figure 2 highlights the difference between image classification, object detection, and image segmentation in the field of vehicle damage assessment.

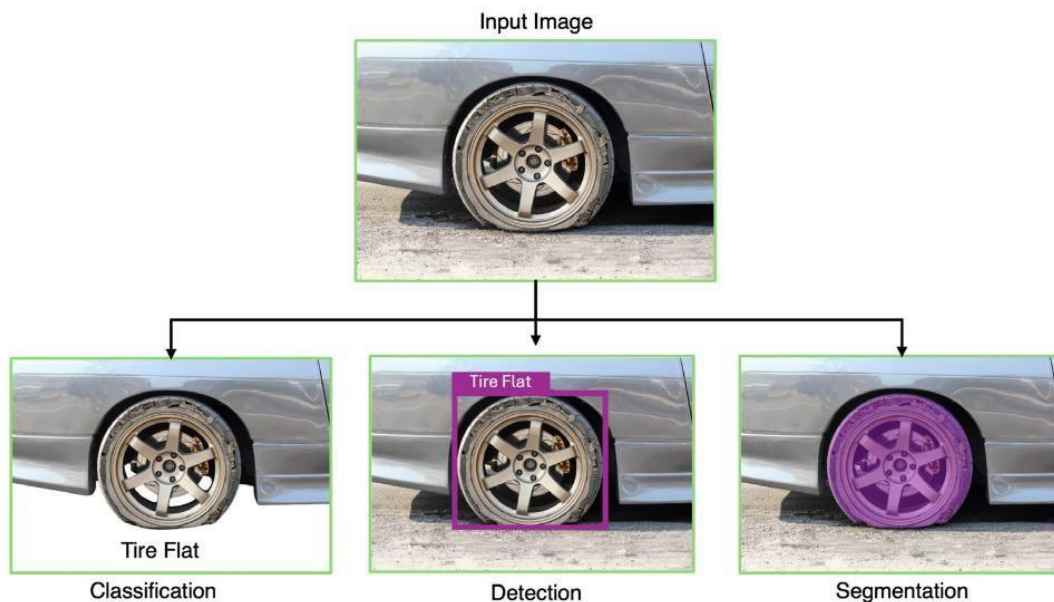


Figure 2: Different types of detecting vehicle using artificial intelligence [3]

There are three categories of object detection methods utilized in modern times, namely classification-based, detection-based, and segmentation-based. The categories provide various degrees of spatial detail. Detection-based techniques provide a good tradeoff between accuracy and speed to be used in real-time vehicle inspection. Also, since a size of detected objects and its location may assist in providing significant contextual information, the object detection approaches may also facilitate other auxiliary tasks such as estimating the cost of repair and the intensity of the damage [3].

2.5.3 Instance Segmentation

Instance Segmentation is a more detailed computer vision task that detects objects and generates a pixel-wise mask for each object instance. The use case with vehicle damage assessment is very helpful, as the shape of the damage is typically not regular and is not easily described with a simple bounding box. The use of instance segmentation in this project is to retrieve masks of damaged areas that enables the system to estimate the relative damaged area which is utilized as a part of the severity estimation procedure.

2.6 Object Detection vs Segmentation for Damage Analysis

Both object detection and segmentation are useful for the Vehicle Damage Analysis, but they offer different amount of detail. Object detection provides bounding boxes indicating the approximate location of damage and segmentation gives a more precise outline of the damaged region at the pixel level. In this study, segmentation is more appropriate as severity estimation is based on the damaged area that can be seen.

Damage to the vehicles frequently occurs in unusual patterns or shapes, particularly in the form of scratches, cracks, dents, and broken glass and this can result in incorrect severity assessments. Therefore, segmentation masks were used as they allow the calculation of a more meaningful damage area ratio and the generation of more realistic damage severity and repair cost outputs.

2.7 YOLO-Based Models

The you only look once (YOLO) object detection algorithm, introduced by Redmon et al. in 2015. Unlike other models who used to search for objects first then classify them, YOLO uses one neural network that looks at the full image at once, then divides it into grids and each grid cell predicts where objects are and what they are [19]. The GoogLeNet model for image classification inspired the network architecture of YOLO models, and it is consisting of three main parts: a backbone network for extracting features, a neck that aggregates features across various scales, and a detection head that predicts both object locations and classes. This specific architecture allows YOLO models to have high detection speed while ensuring competitive accuracy is preserved. Intersection over Union (IoU) and Non-Maximum Suppression (NMS) are key concepts in YOLO-based object detection, where IoU measures the overlay between the predicted and the ground-truth bounding boxes, and NMS is applied to eliminate redundant overlapping predictions and retain the most confident detections. The main YOLO architecture is shown in figure 6.

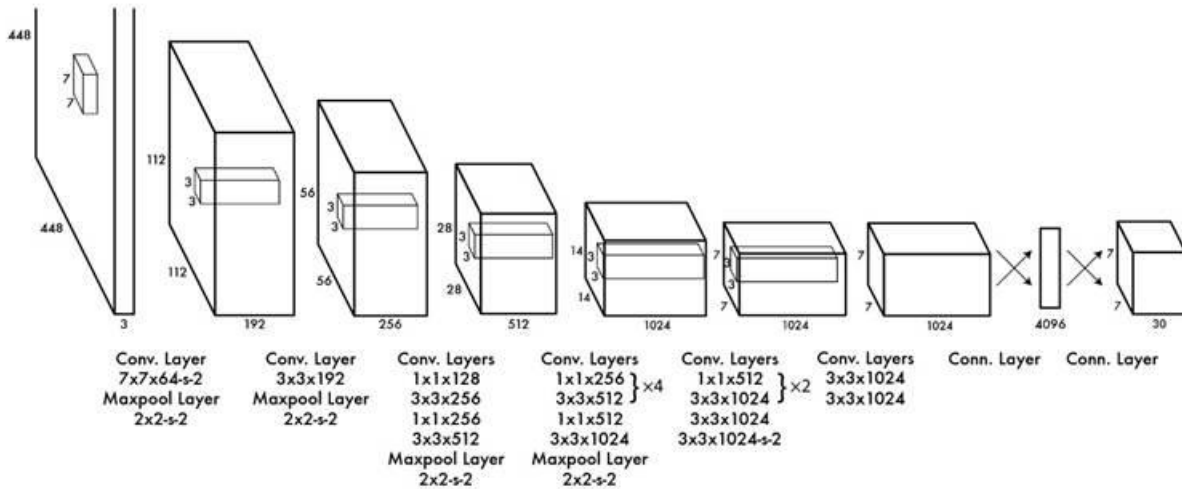


Figure 3: The original architecture of the YOLO object detection framework [19]

2.8 YOLOv11m-seg Architecture

The YOLOv11m is a medium-sized model in the family of object detection and instance segmentation algorithms, which seeks to find a practical trade-off between performance and computational efficiency. In other words, YOLOv11m is well-suited for use in real-world applications because of its ability to provide good results at high performance levels.

There are several advantages offered by the YOLOv11m algorithm. The first major advantage of YOLOv11m is that it always provides better accuracy compared to its lighter counterparts, like nano or small. These models seek to minimize computation time, but at the cost of being unable to handle complicated image scenes or fine-grained visual details, including occlusion and overlapping objects. By using a more powerful architecture and deeper layers, YOLOv11m can better capture relevant information from images, resulting in improved accuracy and generalization across different image types and conditions.

However, on the contrary, YOLOv11m proves to be far more effective than smaller size YOLO models and takes less capacity than larger YOLO models. Even though in certain scenarios, large or extra-large models could produce better results in terms of accuracy, their effectiveness would be outweighed by the massive boost in computational power, storage, and inference time required to train them.

Concerning implementation, YOLOv11m stands out in the sense that it is quite versatile and can be applied easily with medium-level GPU and hardware configurations. That makes it an excellent option for development as well as deployment purposes. Due to these factors, YOLOv11m can be referred to as the ideal middle ground in the field of accuracy.

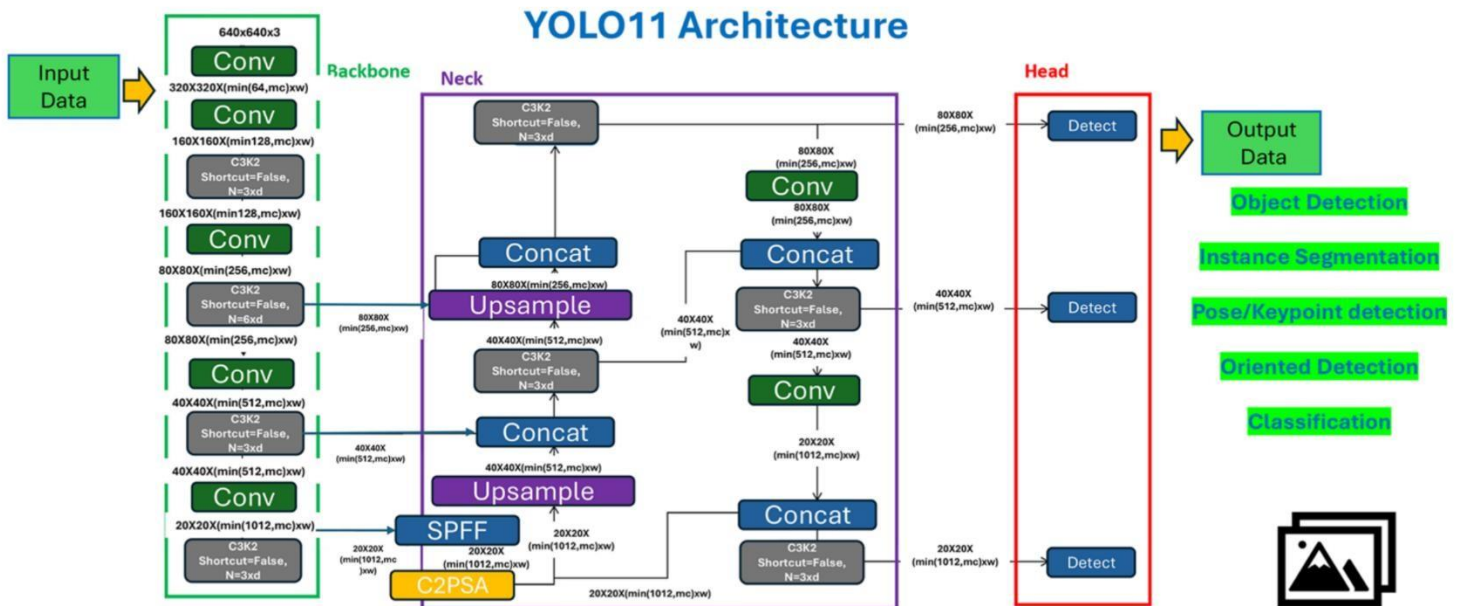


Figure 4: YOLOv11 architecture with backbone, neck, and head components [20]

2.9 Vehicle Damage Types and Visual Challenges

The damage to vehicles may be in the form of surface damage like scratches, structural dents, and in the form of damaged or detached parts. Most of the scratches only affect the surface paint: but the more profound impact of dents involves alterations and deformations of the body and can produce more serious long-lasting impact. Broken materials such as broken headlights, broken mirrors or bent bumpers are generally changed rather than repaired. Nearly all the prevalent types of disfigurement to vehicles such as scratches, dents, and broken parts are illustrated in Figure 3.

Effective assessment and cost estimation of repair are based on the precise identification of these types of damages.



Figure 5: Examples of common vehicle damage types used in image-based inspection [6]

There are however several visual obstacles that can influence the damage detecting of images. Light variations are one of the leading challenges, it may have a bad effect on visibility of the damages by obscuring it or overemphasizing visibility of the damaged part, other challenges include camera angle, where taking pictures of the same damaged part taken with different angles can confuse the system resulting in errors. Occlusions caused by shadows and other objects around

are also some of the factors that hinders visibility and confuse the system. Finally, in other instances, variations in vehicle colour, material, and design make the process of damage recognition difficult as it introduces variability. These visual challenges highlight the complexity of detecting vehicle damage automatically and the need for a dependable AI model that can handle imaging conditions present in the real world.

2.10 Severity Estimation in Vehicle Damage Assessment

Severity estimation refers to the classification of the detected damage based on its apparent level of severity. The severity typically depends on the type of damage, the extent of the damage, and the impact on safety or functionality in the case of vehicle inspection systems. For instance, a small scratch might be a minor issue, whereas a large crack, broken lamp or flat tire may be more pressing.

In this project a rule-based approach is adopted to estimate the severity. The damage is classified into severity levels like minor, moderate or severe based on the detected damage class and the ratio between the segmentation mask area and the full image area. The severity thresholds are designed conservatively because the system does not employ a separate model to segment the entire car body or individual car parts. This minimises the risk of assessing a simple damage as serious damage unless the identified damaged area is obvious or the damage is deemed to be safety critical.

2.11 Repair Cost Estimation in Vehicle Inspection Systems

Estimating repair costs is a significant component of vehicle damage assessment as users often require knowing the potential financial consequences of the damage detected. When damage occurs in the real world, the repair expenses vary based on the severity, the type of damage, the model of the vehicle, and the cost of the spare parts, as well as the price of the work, the labour cost and the availability of the market. So, repair cost estimation needs to be expressed as a range of values, not a specific repair cost.

The approach to repair cost estimation in this project is to use a range of costs for each damage class that have been previously defined. The ranges are correlated with the severity level to offer a more realistic estimation. For instance, the cost of repaired a little scratch is less than that of replacing a crack or a flat tyre.

2.12 Mobile Application-Based Inspection Systems

Mobile application-based inspection systems enable the user to conduct an initial vehicle damage assessment from a distance without having to go to the repair shop/service station or inspection centre immediately. A user can upload an image of a vehicle from his mobile application, get vehicle detection results, get severity info, get cost info and can download a report. This will increase accessibility and shorten the time for a first assessment of damages.

The mobile application is the human aspect of the proposed system in this project. It links the user to the trained damage detection model and displays the result in an understandable and comprehensible format. The application also includes features like login and sign-out, chatbot support, download of reports, feedback collection, etc., making it more complete as an end-to-end inspection support system.

Chapter 3: Literature Review

3.1 Related Work Overview

M.R.S. Ramazhan et al. proposed a car damage evaluation system, which is a smart car, and is based on an improved version of the YOLO algorithm that is combined with image processing methods. The suggested approach had better detection accuracy than typical YOLO baselines with a custom dataset [6].

U. Maheswari et al. described a car damage price prediction model, which combines transfer learning using MobileNet with Fast R-CNN and Mask R-CNN damage localization algorithms, and is aided by a database back-end [16].

D.Z Baig et al. proposed a specialized dataset and deep learning system that aims at identifying small dents on the surfaces of vehicles. The experiment has used YOLOv8 variants and high data augmentation measures. Competitive precision, recall, F1-score, and mAP were demonstrated in the results of the experiment [17].

V. Ghita et al. have developed market-oriented research focused on the use of artificial intelligence to detect the damage in cars and estimate the repair costs in the insurance market. Their research combined the pilot survey of drivers and insurance inspectors and proposed one conceptual AI-based system called “InsureAI”. The results turned out the high demand of the AI-assisted solutions to assist in the evaluation in insurance claims and their decision-making [7].

S.A Perez-Zarate et al. suggested a machine-assessment system of car damage that can be used in the insurance domain based on a group of the YOLOv5 and YOLOv8 object detectors. It was assessed with the help of an original dataset of 7,896 damaged and 2,464 non-damaged images, as well as on the publicly available TartesiaDS dataset. The system obtained steady accuracy, recall and AUC scores with a low rate of false negative [13].

Vijay M. et al. used machine learning techniques for car damage detection using YOLOv8 and Mask R-CNN, though these two models were not used in the developed implementation. This paper was based on the use of manually gathered vehicle damage images and reported acceptable

qualitative results in telling whether the damage was present or not and the severity of the damage [14].

K.R. Sumana explored the robustness and generalization of vehicle damage detection based on YOLOv8 and CNN baseline model. The paper adopted a vehicle damage dataset and proved that YOLOv8 was more successful than CNN model in terms of accuracy, precision, recall, and F1-score performance [15].

V. Mistry et al. suggested a CNN damage-detecting system on a VGG-16 architecture that is trained using transfer learning on an intelligent car. The authors suggested that self-collected vehicle damage images made the process of assessing vehicle damage more effective and dependable than manually assessed inspection [11].

P. Warungse et al. explored the applications of a CNN architecture with a ResNet base in car damage detection with the assistance of transfer learning and image augmentation strategies. These strategies enhanced the performance of the classification system in a systematic way as reported by the authors [12].

R.E.V Ruitenbeek et al. studied the concept of convolutional neural networks in vehicle damage recognition using transfer learning-based object detection as well as object classification models. The accuracy of the dataset used in the study was large, having more than 10,000 labeled images of damages. The detection accuracy was quite similar to the accuracy of human experts in a controlled imaging environment [10].

Ritik Gandhi conducted research on the idea of identifying and assessing the damage and severity of cars using various deep learning networks, such as DenseNet, ResNet, and YOLO, all accompanied by data enhancement. Web scraping of online sources was done to collect the dataset. The research indicated a high damage detection and medium severity classification performance of ResNet and DenseNet, respectively [9].

M. Dwivedi et al. suggested a hybrid deep learning model that integrates pre-trained CNN classifiers with YOLOv3 that are used to detect and localize damage. The system achieved a classification accuracy of about 96.3% and a YOLOv3 mAP of 77.7% using a self-collected and manually annotated dataset [8].

The given table offers a comparative summary of the recent studies focusing on automated vehicle damage detection, classification, and estimation of its cost through artificial intelligence

(AI) and deep learning methods. This field has gained more attention because it has the potential of enhancing efficiency, accuracy and uniformity of insurance claim processing, car inspection, and estimating repairs workflows. In these papers, the approaches to the task have been diversified, including transfer learning convolutional neural networks (CNNs) and modern object detectors (YOLO and Mask R-CNN versions) as well as utility-specific datasets have been used to evaluate the extent to which cars have been damaged.

Table 1: Comparison of prior studies on vehicle damage and cost estimation

Authors	Year	Algorithm/Methods	Dataset	Results	Limitations
M.R.S. Ramazhan et al. [6]	2025	Enhanced YOLO object detection + image processing pipeline.	Custom created vehicle damage image dataset.	Demonstrated improved detection precision over basic YOLO baselines.	Numerical metrics such as accuracy or mAP are not provided.
U. Maheswari et al. [16]	2025	MobileNet transfer learning, Faster R-CNN and Mask R-CNN for localization + SQLite database backend.	Damaged car images uploaded via a web interface.	Achieved diagnostic accuracy for 95% in damage severity classification, as well as reduced false positive and negative rate.	Dataset not publicly available. Limited information about image resolution, annotation and dataset split.
D.Z. Baig et al. [17]	2025	YOLOv8 object detection variants + data augmentation.	A customized dataset of annotated microscopic dents on car surfaces.	Best model precision around 86%, recall 84%, F1-Score 85% and mAP between 50 and 60%.	Slow convergence. Specialized dent focus.
V. Ghita et al. [7]	2024	Market Research + proposed AI system architecture.	Pilot Survey of drivers and insurance inspectors.	Identified strong demand for AI-assisted damage detection. Proposed “InsureAI” framework	Not validated on real damage image datasets. Conceptual architecture only.

S.A. Perez-Zarate et al. [13]	2024	A combination of YOLOv5 and YOLOv8 object detectors with confidence threshold optimization.	A created dataset including 7,896 damaged images and 2,464 undamaged images. Public TartesiaDS with 108 images.	Ensemble of YOLOv5 achieved balanced results of precision, recall and AUC, while keeping the false negative rate around 9%.	High model complexity. Relies on uniform image capture standards. Limited Segmentation categories.
Vijay M. et al. [14]	2024	YOLOv8 and Mask RCNN but not used for final implementation.	A manually collected vehicle damage images.	Satisfactory results for damage detection and severity prediction.	No quantitative metrics are provided.
K.R. Sumana [15]	2024	YOLOv8 for object detection + CNN baseline.	A RoboFlow vehicle damage images dataset.	YOLOv8 model achieved 85.4% accuracy, while the CNN model showed 78%. Also, YOLOv8 showed higher precision, recall, and F1-score values.	Moderate dataset size. The model evaluation focused on classification rather than severity scoring.
V. Mistry et al. [11]	2023	CNN + Transfer Learning (VGG-16 based).	Self-collected vehicle damage images.	Reported improved accuracy and consistency over manual inspection.	No detailed performance metrics were published. Neither dataset size nor splitting were reported.
P. Warungse et al. [12]	2023	CNN with ResNet backbone accompanied with transfer learning and image augmentation.	An annotated vehicle image dataset has been created.	Systematic improvements have been reported in terms of classification performance.	Dataset size and annotation details not clear. Results are not benchmarked against public dataset.
R.E.V. Ruitenbeek et al. [10]	2022	CNN-based object detection and classification using transfer learning strategies.	Dataset with more than 10,000 labelled damage images	Achieved an expert-comparable detection accuracy under controlled conditions.	Performance affected strong reflections. Limited details of real-world deployment.

Ritik Gandhi [9]	2021	Multiple deep learning models including DenseNet, ResNet and YOLO with augmentation.	New selfcollected image dataset via web scrapping -> Google, Bing, etc.	DenseNet accuracy for severity classification around 70% and 96.3% ResNet damage detection accuracy.	Overfitting issues. Limited dataset. Severity classification less stable.
M. Dwivedi et al. [8]	2020	Pre-trained CNN classifiers + YOLOv3 for detection.	A self-collected dataset of vehicle damages which is manually annotated.	Accuracy results of approximately 96.3% and YOLOv3 mAP up to 77.7% for damage localization.	Small custom dataset which raises overfitting concerns.

3.2 Summary and Research Gap Analysis

Even though numerous studies present encouraging outcomes, the discipline continues to be challenged by similar problems that makes benchmarking and application in practice difficult. The reviewed papers prove the effectiveness of AI-based algorithm of detecting the damage to a vehicle; however, they also reveal the sphere of improvement such as the standardization of datasets, strong generalization in the real-life scenario, computational efficiency, and embedding into the system of insurance and inspection.

In a study where researchers are analyzing automated vehicle damage detection, they use image datasets to train, validate and test AI models. The broad classification of these data could be made as either public or private, and each type has its benefits or drawbacks. While public datasets allow reproducibility comparison between studies, there are a limited number of publicly available datasets of vehicle damage. Most research gather their own data to capture certain vehicle models, types of damage or insurances. Although custom datasets allow them to develop models specially, they may not be diverse or be fully annotated, which can cause them to be biased.

Chapter 4: Methodology

4.1 System Overview

This chapter presents the methodology adopted to develop the smart vehicle inspection and repair cost estimation system. The proposed system takes vehicle images as input and processes them using deep learning–based instance segmentation techniques to identify damaged parts, estimate the damage severity level using the predicted mask area ratio and match the damage detected with suitable estimated repair cost. The system is designed to operate through a mobile application, where users upload images of their vehicle and the system uses the AI-based model to detect the damaged location then links it with estimated repair price and presents it to the user in a user-friendly format. The general build of the suggested system, including user interaction and backend processing components, is shown in Figure 4.

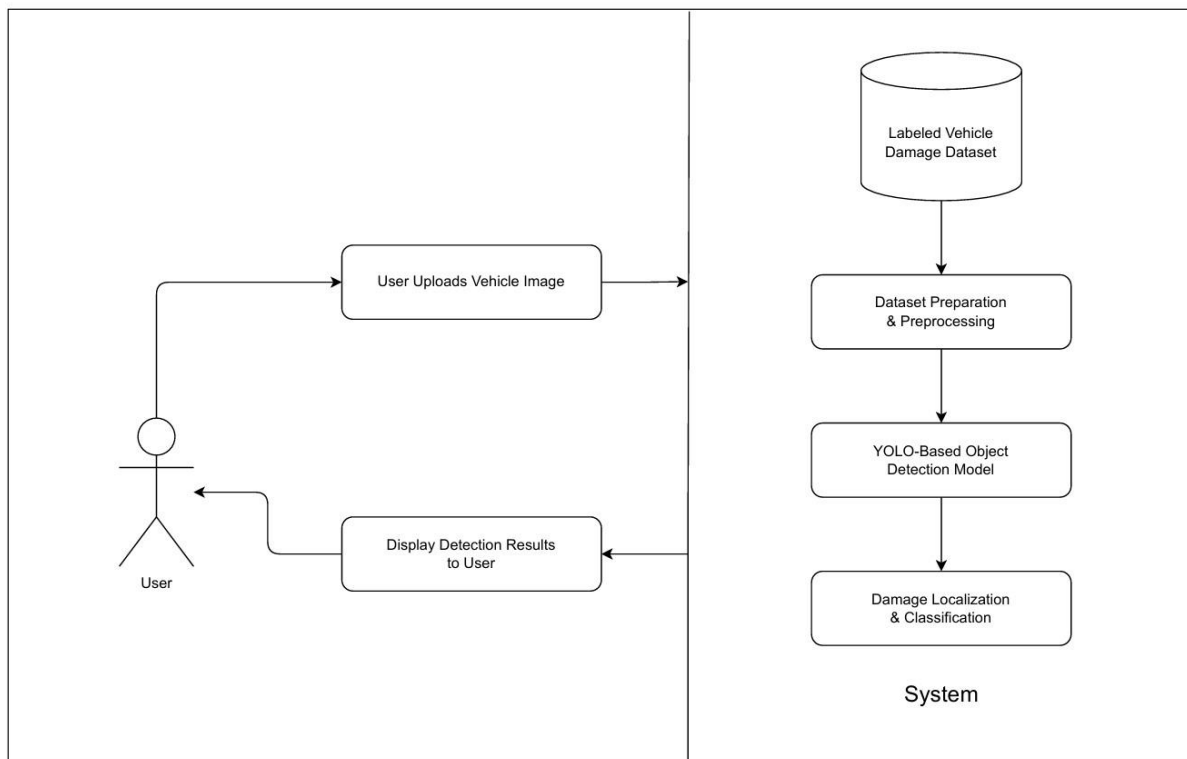


Figure 6: System Architecture of the Proposed Vehicle Damage Detection Application

4.2 Dataset Description

4.2.1 Car Damage Detection Dataset

The Car Damage Detection Dataset (CarDD) [21] is a large-scale publicly available benchmark dataset developed for automated vehicle damage analysis using deep learning techniques. It consists of approximately 4,000 real-world images of damaged vehicles collected under diverse and unconstrained conditions, including variations in lighting, weather, background complexity, camera viewpoints, and vehicle models. The dataset provides a total of 9,163 annotated damage instances, making it a rich and comprehensive resource for training and evaluating computer vision models. These annotations are distributed across six primary damage categories: dent, scratch, crack, glass shatter, broken lamp, and flat tire, covering a wide spectrum of both minor cosmetic defects and more severe functional or structural vehicle damage. Each damage instance is annotated at the pixel level, enabling the dataset to support instance segmentation, object detection, and fine-grained localization tasks with high precision.

The major advantage associated with the CarDD dataset is the presence of variability and realism in the images because the dataset was collected from real-world scenarios as opposed to artificial data or laboratory environments. It poses difficulties that include occlusions, reflections, complex backgrounds, and inconsistent lighting, all of which are similar to what would be seen in a practical setting where vehicle damage is concerned. The dataset also contains numerous small-scale and hard-to-detect instances of damage, making the task of detecting the same extremely challenging, hence encouraging the building of more accurate damage detection frameworks. The size of the damages in the image is largely unbalanced, with most of them occupying very small portions of the picture.

4.2.2 Repair Price Dataset

The repair cost data used in this study has been created specifically to offer realistic figures in regard to the repair costs of various types of damage identified by the proposed system. This data set has been collected using a number of sources including field surveys as well as consulting professionals from various auto repairs workshops. In terms of collecting information, survey forms have been distributed among individuals belonging to diverse backgrounds. Moreover, visits have also been paid to certain automobile repair shops in order to gain professional insight into repair cost estimates of various types of vehicle damage.

The data set includes damage types that are often recognized through the damage detection model, such as scratches, dents, cracks, broken glass, broken lights, and tire damage. The minimum, average, and maximum repair costs for every damage type have been taken into account to cover the differences in repair difficulty, labour costs, and replacements. Additionally, any changes in cost due to severity have also been included.

8. What was the approximate cost of the repair? (of the part you selected in question 7)

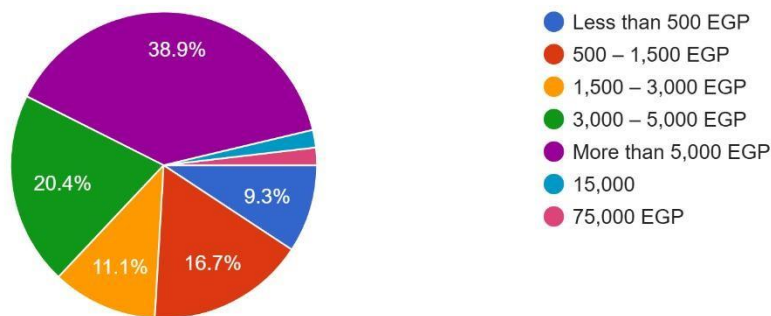


Figure 7: Distribution of approximate vehicle repair costs reported by survey respondents

4.2.3 FixZone Chatbot Dataset

The chatbot database has been created with the intention of serving the FixZone Damage Assessment System by offering structured replies for different types of queries that may arise in connection with vehicle damage assessment and repair services. The chatbot database includes questions from users along with the respective categorization and appropriate response from the chatbot. Using the similarity index, the chatbot recognizes the intent of the question and offers a response based on the category. Some of the major areas covered by the chatbot database include greeting, gratitude message, damages vehicle parts, damage assessment, costs estimation, workshop location, final report of assessment and general system services. To make the chatbot database more effective in recognizing various versions of user queries within the same category, the database includes multiple versions of user questions for different categories.

Such methodology will guarantee that all the interactions will be reliable and accurate, avoiding possible misinterpretations and providing irrelevant answers. This way, the collection of

data becomes a significant part of the FixZone system and provides an opportunity for the chatbot to help users navigate, answer frequent questions, inform users regarding damage on their car, etc.

4.3 Data Preparation and Pre-processing

The CarDD data is formatted in the YOLO segmentation format. In this format, every image is associated with a label file that includes the polygon coordinates and class label for the damaged area. These polygon annotations are used to train the model to make predictions on segmentation masks for every identified damage instance.

The dataset was then split into training, validation and test sets and all the images were resized to 832×832 pixels during the training phase. The image size was chosen to maintain a sufficient amount of visual information in the damaged areas, such as scratches and cracks, while still maintaining reasonable computational requirements for training. Augmentation and preprocessing steps were used to train the model, as these operations aid in the generalization of the model in various imaging conditions. This is particularly important as real vehicle damage images can differ in lighting, background, viewing angle, color and damage size.

4.4 Model Selection and Justification

Since the proposed system is intended to identify the type of damage, locate the damaged region, and generate a mask of the damaged area at a pixel level, a model that is able to accomplish all three is needed. For this reason, the model chosen for this task was an instance segmentation model, rather than a classification-only or bounding-box-only detection model.

In this project, image classification will not be enough since it will only label the image as a whole and cannot indicate where the damage is. Object detection will provide Bounding Boxes which may contain a large portion of the vehicle surface that is not damaged, resulting in less accurate vehicle surface severity determination. Therefore, instance segmentation is more appropriate as it can yield a pixel-wise mask that more closely represents the shape of the damaged area.

The final model was chosen as YOLOv11m-seg, which can perform both object detection and instance segmentation. The medium version is selected as it is the balance in terms of accuracy and computational cost. The smaller YOLO versions could be more efficient, but they might not detect smaller and finer damage patterns, and the larger versions could need more memory, longer

training time, and more computational power. It was determined that YOLOv11m-seg would be appropriate for this project as it is capable of detecting and segmenting the visible damage to vehicles and is also practical for integration in later applications.

4.5 Model Architecture and Training Configuration

4.5.1 Training Configuration

The model YOLOv11m-seg was trained through transfer learning, which means that the pre-trained weights were used as the initial parameters before fine-tuning to adapt to the vehicle damage dataset. This particular model was set up for the instance segmentation problem. That meant that it was capable of performing both object detection and segmenting pixel-level masks of damaged areas on the car. For achieving greater reproducibility and consistency during training, a random seed of 42 was used and deterministic training was enabled.

Images with dimensions of 832×832 pixels were used for training the model. This way, the model could learn more about visual information without taking much time. The number of epochs and batch size were set to 100 and 4 respectively. To avoid overfitting and additional training steps, early stopping with a patience of 25 epochs was used.

During training, the model was constantly training itself on feature representations relating to various categories of car damage as well as developing its segmentation skills. The training process involved an automated checkpoint creation, allowing the saving of both optimal and final models for further use and testing purposes. Besides, numerous training and validation results were produced in terms of various graphs such as performance tracking diagrams, learning curves, confusion matrices, and others. These artifacts were saved for further interpretation, which is described in more detail in the Results and Evaluation chapter.

The key training configuration in this project is summarized in Table 2.

Table 2: YOLOv11m-seg training configuration

Parameter	Value
Model	Yolov11m-seg
Task	Instance Segmentation
Training Approach	Transfer Learning
Input Image Size	832 x 832
Number of Epochs	100
Batch Size	4
Early Stopping Patience	25
Training Platform	Google Colab

4.5.2 Model Workflow

The YOLOv11m-seg-based vehicle damage detection system follows a structured workflow from data preparation to final prediction.

Step 1: Dataset Preparation

The dataset is collected, annotated, and split into training, validation, and testing sets with segmentation masks for different damage types.

Step 2: Data Preprocessing

All images are resized to 832×832 pixels and formatted according to the YOLO segmentation requirements.

Step 3: Model Initialization

The YOLOv11m-seg model is initialized with pre-trained weights using transfer learning to adapt it to the vehicle damage task.

Step 4: Model Training

The model is trained using defined hyperparameters such as batch size, epochs, and early stopping, learning to detect and segment damaged regions.

Step 5: Validation

Performance is monitored on the validation set using metrics such as loss curves, precision-recall graphs, and confusion matrix.

Step 6: Model Saving

The best-performing weights are saved during training for later use.

Step 7: Inference

The trained model is used to predict vehicle damage on unseen images, producing bounding boxes and segmentation masks.

Step 8: Post-Processing

Predictions are refined using confidence thresholds to remove weak or incorrect detections, producing the final output.

4.6 Evaluation Metrics

Commonly used object detection and instance segmentation methods were used to evaluate the trained model. To evaluate the model's performance in identifying the correct damage detection, the damaged region localization, and segmentation mask generation, these metrics were used.

Precision measures the proportion of predicted damage instances that are correct. High precision value indicates fewer false positive detections by the model. Recall is the percentage of damage events that the model is able to identify. A high recall value is when the model fails to miss out in recognizing more damage cases. The F1 score is a combination of precision and recall and comes in handy when assessing the trade-offs between false positives and false negatives.

Intersection over Union (IoU) is the ratio of the size of the intersection to the size of the union. When using the bounding boxes, IoU is computed between the predicted and the actual boxes. It compares the segmentation mask predicted by the model with the ground-truth mask for masks. The fixed overlap threshold was used to evaluate at $\text{mAP}@0.5$. The stricter $\text{mAP}@0.5:0.95$ metric was also taken into account since it takes the average into account at several IoU thresholds from 0.5 to 0.95.

The evaluation was conducted independently for the bounding-box detection and the segmentation-mask. The distinction is relevant because a model might classify the correct class and approximate location but have poorer mask boundaries. The following table is a summary of the evaluation metrics that were used in this project.

Table 3: Evaluation metrics used for model assessment

Metric	Description
Precision	Measures how many predicted damage instances are correct
Recall	Measures how many actual damage instances are detected
F1-Score	Harmonic Mean of precision and recall
IoU	Measures overlap between prediction and ground truth
mAP@0.5	Mean average precision at IoU threshold 0.5
mAP@0.5:0.95	Mean average precision averaged across IoU thresholds from 0.5 to 0.95
Confusion Matrix	Shows correct and incorrect predictions per class

4.7 Post-Processing and Confidence Threshold Selection

YOLOv11m-seg generates raw predictions after the inference process, which include the predicted damage class, confidence score, bounding box, and segmentation mask. Some raw predictions are not accurate though. In some instances, low confidence predictions might be false positives, particularly in the case of reflection, shadowing or surface characteristics that look like scratches or dents in the image.

For better reliability of final result displayed, the use of confidence thresholding was done during post processing. Predictions with confidence scores less than the chosen threshold were filtered out. The filtering module removes the false or low-quality detections before they are fed into the severity and repair cost estimation module.

Class specific confidence thresholds were considered, as damage classes have different characteristics and degrees of difficulty. The example of the classes, such as glass shatter, broken lamp, and flat tire, are all visually distinctive classes where a higher level of confidence can be found and the example of the scratch, dent, and crack classes are subtle and may demand more careful consideration of the threshold selection. The last thresholding scheme was designed to

minimize the number of false detections while preserving the number of true detections essential to the application.

Although confidence thresholding improves the reliability of displayed predictions, it may reduce recall because some true damage instances with low confidence scores can be removed during filtering. But for an application that is meant for the user, fewer and more reliable detections may be more useful than many uncertain predictions.

Table 4: Class-specific confidence thresholds

Damage Class	Final Confidence Threshold
Dent	0.35
Scratch	0.35
Crack	0.25
Glass Shatter	0.30
Broken Lamp	0.40
Flat Tire	0.40

4.8 Severity Estimation Methods

A rule-based method was employed for severity estimation with two major inputs: predicted damage class and the area of segmentation mask. The aim of this module is to categorize the detected damage into three severity categories: minor, moderate and severe. For each detected damage instance, the predicted segmentation mask was used to calculate the damaged area. This was achieved by calculating the number of pixels that are assigned to the predicted mask. The damage ratio was then computed as the ratio of mask area to the image area. So, damage ratio is equal to the area of the damage mask predicted by the model divided by the area of the full image.

The ratio provides a good approximation of the portion of the image that is affected by the damage. This ratio was compared to class specific severity thresholds. Different thresholds were applied for each class, since each class represents a different type of damage with a different level

of meaning. A small scratch is typically not as bad as a flat tyre or a broken lamp, although both are confined to a small area of the image.

The severity rules were defined conservatively as there is no separate car-part segmentation model in the system. This implies the system does not know the precise dimensions of the damaged part of the car, like the bumper, door, lamp or windshield. So, the severity was scored based on the ratio of the damage area to the full image area, but not on the ratio of the damage area to the specific part of the car. This limit was taken into account in designing the thresholds to minimize the possibility of overestimate of cosmetic damage.

The severity thresholds are listed in the table below:

Table 5: Class-specific severity thresholds

Damage Class	Minor to Moderate threshold	Moderate to Severe threshold	Additional Rule
Scratch	0.030	0.180	Severe only if widespread
Dent	0.025	0.120	Severe only if visibly large
Crack	0.020	0.100	Serious, but not instantly severe
Glass Shatter	0.040	0.200	Severe if large glass region is affected
Broken Lamp	0.030	0.150	Minimum severity is moderate
Flat Tire	0.000	0.000	Always Severe

The final severity level was determined by the following rule:

- Damage was considered minor, when the damage ratio was below first threshold.
- Damage was defined as moderate if the damage ratio was in between the first and second threshold.
- Damage ratio exceeding the second threshold was considered as severe damage.

The damage class was taken into account in the damage severity logic, in addition to the damage ratio. More serious damage types, such as safety related or functional damage like broken lamps or flat tires, may need more serious treatment than cosmetic damage like scratches. Thus, if a broken lamp is classified as minor by area, it is upgraded to moderate because it affects safety.

4.9 Repair Cost Estimation Method

The repair cost estimation module was designed as a rule-based system which connects each damage class identified with an approximate range of repairing cost. This module aims to give the user an initial estimate of the likely repair costs following the damage has been identified and the extent has been estimated.

Min and max cost for each damage class was established through survey responses and workshop information on repairs. The severity level was then used to adjust the estimated cost within the corresponding range. The lower part of the range was associated with minor damage, the middle part of the range was associated with moderate damage, and the higher part of the range was associated with severe damage.

The cost range for the various repairs in the proposed system is given in table below:

Table 6: Approximate repair cost ranges used by the system

Damage Class	Minimum Cost (EGP)	Maximum Cost (EGP)
Scratch	300	2,000
Dent	600	3,000
Crack	1,200	2,200
Glass Shatter	750	2,000
Broken Lamp	1,500	2,300
Flat Tire	10,000	20,000

The Repair Cost module does not provide an exact quotation for repairs. These actual repair costs may differ from car to car, part to part, brand to brand, and even from one workshop to another due to the market. Hence the cost estimation result is a preliminary guide and should be used as a reference to help the user before professional inspection.

4.10 Chapter Summary

The methodology involved in the development of the proposed smart vehicle inspection and repair cost estimation system was described in this chapter. A YOLOv11m-seg instance segmentation model was trained and tested on the CarDD dataset to identify and segment six classes of vehicle damages. The chapter also clarified the data preparation method, model selection, training configuration, evaluation metrics, post-processing strategy, severity estimation method, repair cost estimation method and mobile application methodology.

The proposed method is a full pipeline, from uploading images to providing an easy-to-use inspection result. The use of segmentation masks enables getting an estimation of the visible damaged surface, and the rule-based severity and cost modules transform the model output into concrete useful information for the user. The implementation environment, experimental results, model evaluation, severity and cost outputs, application tests and system performance discussion are presented in the next chapter.

Chapter 5: Results and Comparisons

5.1 Implementation Environment

The experimental implementation was done using Google Colab, the proposed vehicle damage detection and segmentation system was developed using Python and the Ultralytics YOLO framework. Model training, validation, testing, inference, post-processing, severity estimation, and repair cost estimation were performed in the same environment.

To speed up the training and evaluation process, an NVIDIA A100 GPU was used. PyTorch was used as the deep learning backend for training the YOLOv11m-seg model. Several Python libraries were also used during implementation. OpenCV was used for image processing and visualization, NumPy was used for numerical calculations, Pandas was used for handling tabular results and CSV files, and Matplotlib was used for plotting graphs, visualizing the training, and testing results.

The training configuration, including the model type, image size, number of epochs, batch size, and early stopping settings, was already explained in Chapter 4. Therefore, this chapter focuses on the dataset split used for evaluation, the training behaviour, validation results, test performance, confusion matrix analysis, post-processing results, severity and repair cost estimation results, and the overall discussion of the final system.

5.2 Dataset Split and Class Distribution

The CarDD dataset was used as the main dataset for training, validation, and testing. The dataset was converted from COCO annotation format to YOLO segmentation format before training. In YOLO segmentation format, each image is associated with a label file that contains the class label and polygon coordinates of the damaged region. This format allows the model to learn both the location and the segmentation mask of each damaged instance.

The dataset was organized into three main subsets: training, validation, and testing. This split follows the official CarDD experimental setup, where the dataset is divided into 2816 training images, 810 validation images, and 374 testing images, corresponding to approximately 70.4%, 20.25%, and 9.35%, respectively. The training set was used to train the YOLOv11m-seg

model, the validation set was used to monitor the model performance during training and select the best weights, and the test set was used as the final held-out evaluation set.

The class distribution across the dataset is shown in Table 7. The distribution shows that scratch and dent have the highest number of instances, while crack, glass shatter, lamp broken, and tire flat have fewer samples. This is important when analyzing the final results because the number of training examples and the visual nature of each damage class can affect the model’s ability to detect and segment the damage accurately.

Table 7: Class distribution across the CarDD dataset splits

Damage Class	Training Instances	Validation Instances	Test Instances
Dent	1806	501	236
Scratch	2560	728	307
Crack	651	177	70
Glass Shatter	475	135	71
Lamp Broken	494	141	69
Tire Flat	225	62	32

The class distribution also helps explain some of the later results. For example, crack has fewer instances than dent and scratch, and it is also visually thinner and harder to detect. In contrast, glass shatter, lamp broken, and tire flat have more distinctive visual features, which makes them easier for the model to identify even with fewer instances.

5.3 Training Performance and Loss Curves

The YOLOv11m-seg model was trained for 100 epochs with a batch size of 4 and an input image size of 832×832 pixels. Bounding box loss, segmentation loss, classification loss, distribution focal loss, precision, recall, mAP50, and mAP50-95 were among the performance measures and loss curves produced during training.

Throughout the training phase, the box loss, segmentation loss, classification loss, and DFL loss curves all steadily declined. This suggests that the model gradually learned the visual patterns

of the vehicle damage classes. The training and validation curves followed a similar trend, which suggests stable learning behaviour. By the end of training, the box loss had dropped from about 1.53 in the start to about 0.62. Likewise, the segmentation loss dropped from roughly 3.12 to about 1.18. This demonstrates that during the training procedure, the model's object localization and mask prediction both improved.

The performance metrics also increased as training progressed. Precision and recall increased steadily, reaching approximately 0.80 and 0.74, respectively. The mAP50 value for bounding box detection reached approximately 0.77, while the mask mAP50 reached approximately 0.76. The mAP50-95 values were lower than mAP50, which is expected because mAP50-95 evaluates the model across stricter IoU thresholds. Overall, the training curves show that the model learned consistently and did not show a clear sign of severe overfitting.

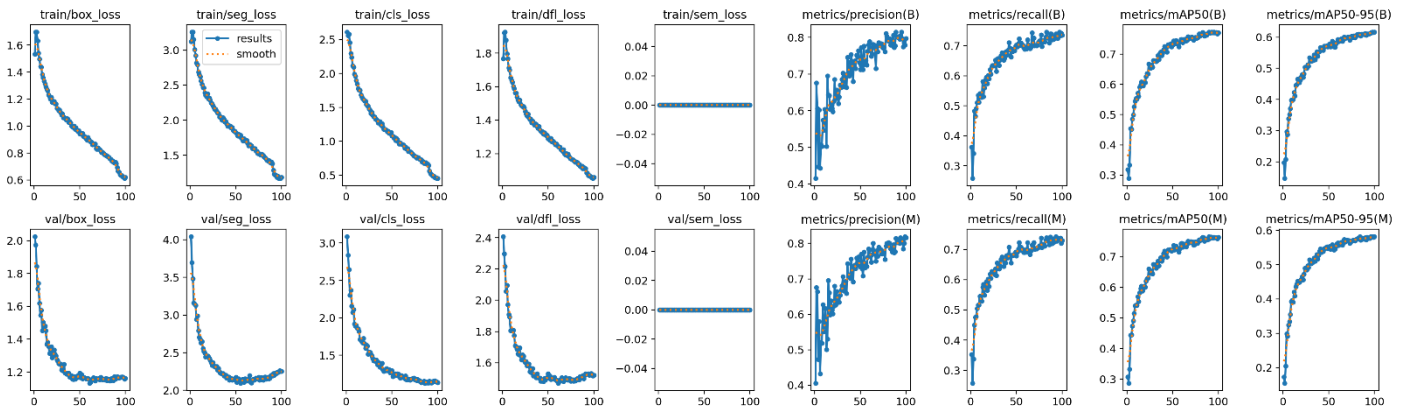


Figure 8: Training loss curves and performance metrics of the proposed YOLOv11m-seg model

5.4 Validation Results

The trained YOLOv11m-seg model was first evaluated on the validation set, which contained 810 images and 1744 annotated damage instances. The official validation metrics were generated using the saved best model weights and the Ultralytics YOLO validation function. The results included both bounding box metrics and segmentation mask metrics.

The model achieved in box mAP50 of 0.770 and mask mAP50 of 0.763. Precision and Recall scores for the Overall box was 0.798 and 0.735 respectively and for the Mask were 0.816 and 0.730 respectively. Overall, differences between the different visible and structurally confirms types of damage outcomes are evident at the class level. Glass Shatter showed great results in terms of model accuracy, with box mAP50 of 0.995 and mask mAP50 of 0.995, implying that shattered

glass creates a distinctive pattern that can be reliably discerned from the model. Tire flat and lamp broken performed well too, closely matching 0.962 and 0.907 box mAP50 scores and similar mask scores of 0.962 and 0.906 respectively, bring up the fact that the model is able to give good quality segmentation masks.

However, the box mAP50 scores of 0.623 and 0.627 and the mask mAP50 scores of 0.610 and 0.597 for dent and scratch were significantly lower. Due to inherent difficulty of both classes, and from background surface variation, these two classes are hard to distinct from each other and background surface variation, as discussed in the confusion matrix below. As for the classes themselves, the mAP50 value of 0.509 (0.508 for segmentation) is below average for crack, which is typical as it is a linear damage nature, and it was the class with the lowest number of instances (177 in val).

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	Mask(P	R	mAP50	mAP50-95)
all	810	1744	0.798	0.734	0.771	0.617	0.816	0.731	0.764	0.581
dent	352	501	0.695	0.559	0.624	0.361	0.715	0.541	0.612	0.327
scratch	431	728	0.676	0.57	0.627	0.361	0.684	0.547	0.598	0.305
crack	122	177	0.599	0.525	0.509	0.308	0.671	0.548	0.51	0.207
glass_shatter	134	135	1	0.972	0.995	0.955	1	0.971	0.995	0.951
lamp_broken	139	141	0.877	0.858	0.907	0.8	0.882	0.858	0.907	0.777
tire_flat	59	62	0.943	0.919	0.962	0.917	0.947	0.919	0.962	0.92

Figure 9: Validation performance of YOLOv11m-seg on the CarDD validation set

5.5 Testing Results

The model was tested on a held-out test set, which contained 374 images and 785 annotated damage instances, and this data has never been seen either during the training or the validation phase. The validation result yielded similarly good box mAP50 (0.769) and mask mAP50 (0.763), indicating a good generalisation and lack of overfitting on the validation data set. Box precisions and recalls on validation set were 0.803 and 0.731, respectively, and box precisions and recalls on validation set were 0.804 and 0.732, respectively. The similarity between the validation and test results indicates that the model generalized well to unseen data and did not depend only on the validation set.

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	Mask(P	R	mAP50	mAP50-95)
all	374	785	0.803	0.731	0.769	0.614	0.804	0.732	0.763	0.575
dent	157	236	0.693	0.614	0.622	0.372	0.707	0.627	0.636	0.35
scratch	183	307	0.673	0.596	0.633	0.358	0.669	0.593	0.607	0.298
crack	48	70	0.568	0.5	0.55	0.328	0.568	0.5	0.53	0.232
glass_shatter	71	71	0.972	0.969	0.989	0.948	0.972	0.969	0.989	0.924
lamp_broken	65	69	0.977	0.841	0.901	0.778	0.977	0.841	0.901	0.768
tire_flat	31	33	0.933	0.864	0.916	0.8	0.933	0.864	0.916	0.876

Figure 10: Test performance of YOLOv11m-seg on the CarDD test set

Like the validation results, the best class scores were at the glass shatter box and mask mAP50 with 0.989 and the tire flat with 0.916, and the lamp broken with 0.901. While dent scored 0.622 (box) and 0.636 (mask), scratch scored 0.633 and 0.607, and crack remained the weakest at 0.550 and 0.530. Although crack performance was lower than the other classes, the result is still understandable because cracks are often thin, narrow, and difficult to distinguish from reflections, shadows, or vehicle body lines.

5.6 Confusion Matrix Analysis

A confusion matrix serves as an elaborate representation of how well the new YOLOv11m-seg model is able to classify images into one of the six possible classes for vehicle damage. Most

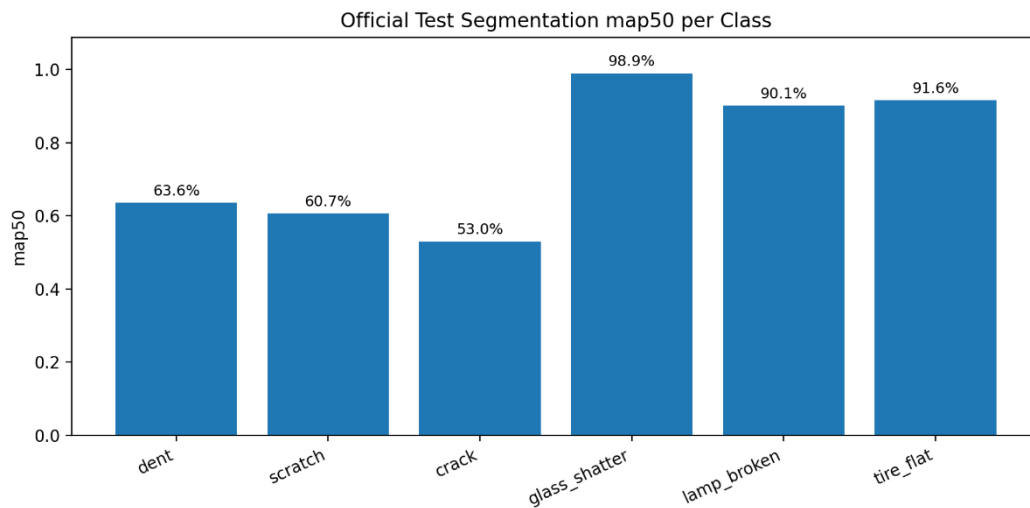


Figure 11: Test performance metrics of the YOLOv11m-seg model

of the predictions are found along the primary diagonals of both raw and normalized confusion matrices, which suggests that the model has managed to correctly classify a huge number of damaged vehicles. In other words, the model has successfully recognized the features of various kinds of vehicle damage. In this matrix, the model correctly detected 159 dent instances, 202 scratch instances, 40 crack instances, 69 glass shatter instances, 58 lamp broken instances, and 28 tire flat instances.

The confusion matrix confirms the same behaviour shown in the mAP results. The model performed best on visually distinctive damage classes, such as glass shatter, lamp broken, and tire flat. These classes have clearer shapes and stronger visual patterns compared to surface-level damage. Glass shatter had the highest normalized true positive rate, with approximately 97% of

actual glass shatter instances correctly detected. Tire flat also achieved a strong true positive rate of approximately 92%, while lamp broken achieved around 88%. This indicates that the model was confident in detecting these classes because they have specific visual characteristics.

The main errors occurred in the dent, scratch, and crack classes. Dent achieved a normalized true positive rate of approximately 63%, while around 32% of dent instances were classified as background. Scratch achieved around 65%, with approximately 33% of scratch instances classified as background. Crack achieved the lowest true positive rate at approximately 59%, with around 40% of crack instances classified as background.

This pattern demonstrates that missed detections when little damage was categorized as background, in addition to confusion between dent, scratch, and crack, were the primary challenges. This makes sense because dents, scratches, and cracks are frequently tiny, thin, or blend in with the car's surface. Reflections, shadows, shifting lighting, and intricate vehicle body texturing may also have an impact. Consequently, the confusion matrix confirms that the model is more accurate for structural and visually obvious damage types, but it is still more difficult for subtle surface-level damage.

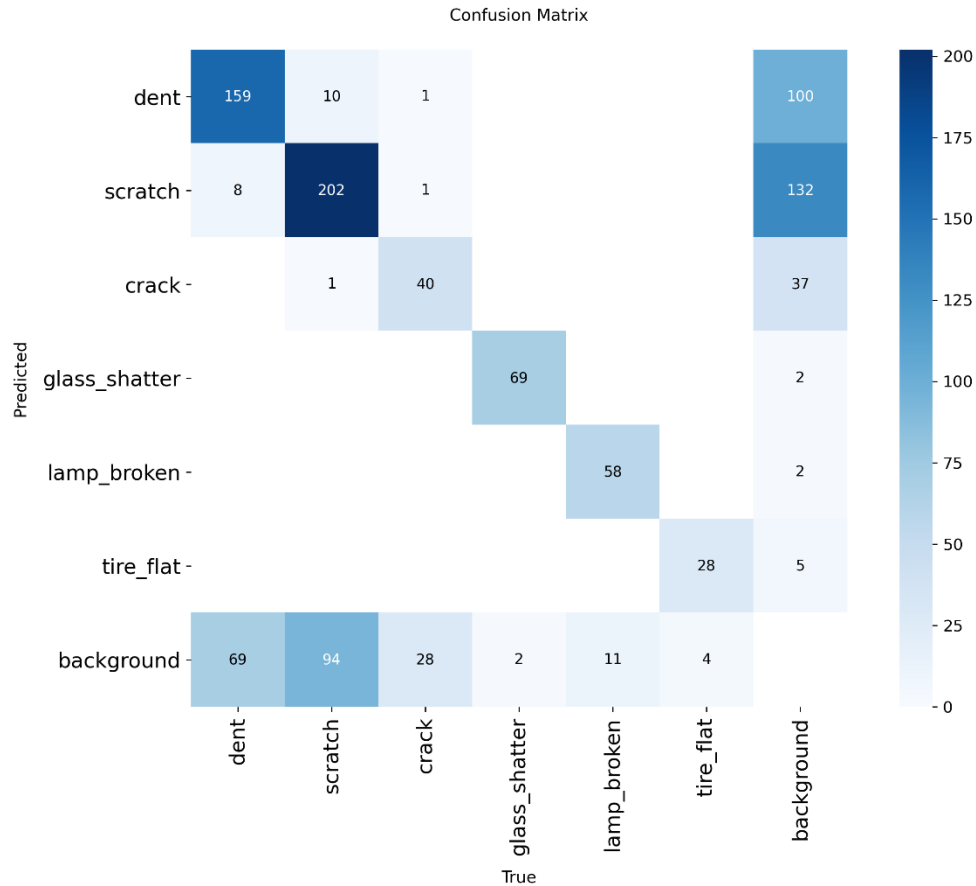


Figure 12: Test confusion matrix of the proposed YOLOv11m-seg model

5.7 Class-wise Performance Analysis

An important measure of the overall detection and segmentation performance is mAP50, which refers to the mean Average Precision at an IoU threshold of 0.50. The box mAP50 and mask mAP50 obtained on the validation set were 0.770 and 0.763, respectively. On the test set, the results were very similar, with a box mAP50 of 0.769 and a mask mAP50 of 0.763. This small difference indicates that the model generalized well to unseen data.

The range of mAP50 results can be quite large per class level and mirrors the difficulty of the available visual examples of each class. Across all tests, glass shatter had a near perfect mAP50 (0.995) on each of the two sets of data and on both box and mask. Lamp broken had a validation tie for the last place with Tire flat at 0.962 and 0.916, respectively, and a close tie-on test with the same at 0.907 and 0.916, respectively. The three classes have unique visual appearances, namely structural discontinuities that are easy to identify and segment in the form of radiating fracture

patterns for shattered glass and the recognisable shape of a flat tyre, and the structural boundary, recognisable by the broken lamp image.

In comparison, Dent and scratch obtained 0.623 and 0.627 on the validation box mAP50 as well as 0.622 and 0.633 on the test set. These 'moderate' scores are natural results of the optical complexity of surface deformations which can change its appearance significantly between different lighting and easily fall to being mis-detected by the background texture. Crack is the lowest performing class (box mAP50 validation and test 0.509 and 0.550, mask mAP50 0.508 and 0.530). The class is a fine linear geometry, with thin cracks being minor "pixels" within the overall image, which can explain the low crack performance, and has been poorly represented by its data, with the total number of thin cracks in the 177 in the validation set and 70 in the test set.

5.8 Test Results Analysis

Figure 13 demonstrates a real-world data, a car image that is not from the dataset and it was employed in order to test the capabilities of the trained YOLOv11m-seg network. This test image includes multiple visible spots with different types of damages (dents and scratches) on the body of the vehicle. Since this test image does not belong to the training dataset, it provides a good illustration of the model's capacity for generalization. In particular, the model managed to analyze this image and make predictions concerning the spots with various damages.

The predicted bounding boxes show the effectiveness of our approach for the task of vehicle damage detection. Indeed, the bounding boxes were accurately drawn around the damaged spots detected in the vehicle, marking all the spots in the vehicle where there was visible damage to it. Our approach succeeded in detecting the spots in the image where there was some damage to the car, as opposed to the normal parts of the car which have not been damaged.

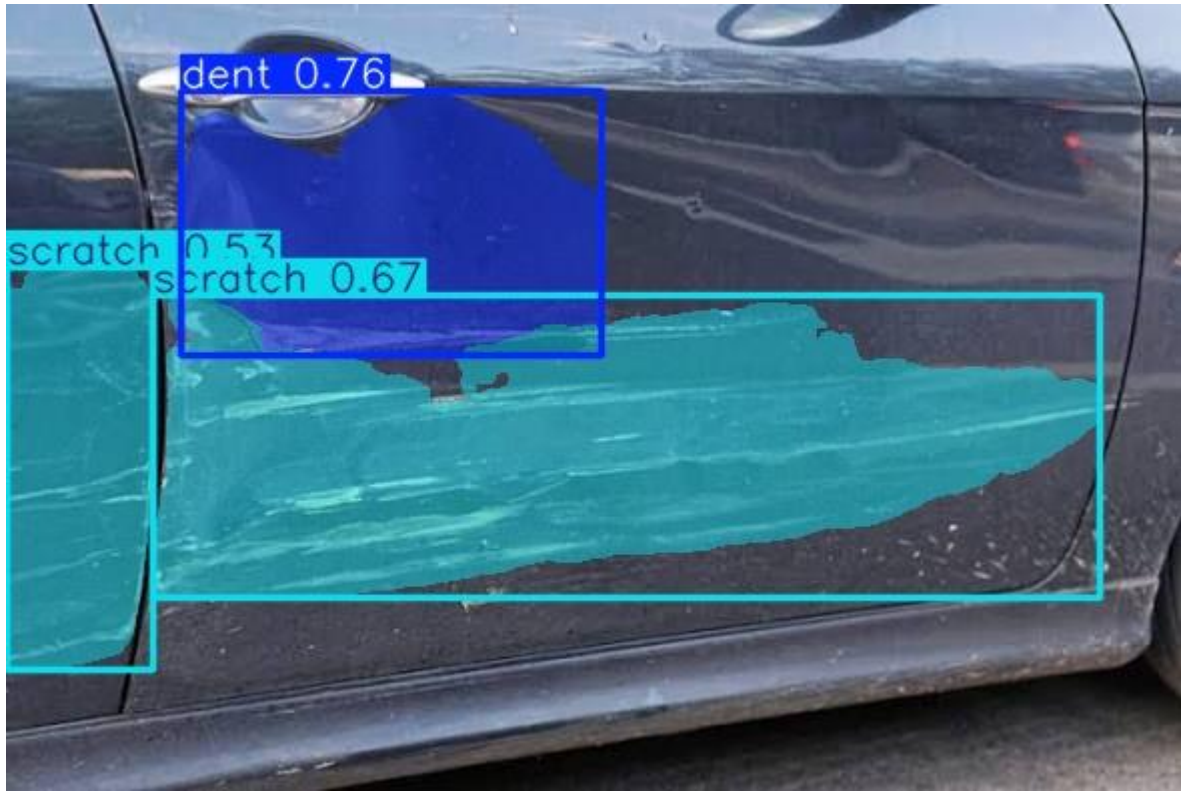


Figure 13: Real-World Vehicle Damage Detection and Localization Result

5.9 Post-Processing Results

After the official YOLO evaluation, post-processing was applied to the raw predictions before passing them to the severity and repair cost estimation modules. The class-specific confidence thresholds described in Chapter 4 were used to remove weak detections, while mask-area filtering was used to remove unrealistic masks. This step was important because the final mobile application should display fewer but more reliable detections to the user.

Table 8: Summary of post-processing effect on test predictions

Stage	Number of Detections	Purpose
Raw YOLO predictions	879	Initial detections produced by the model before filtering
Final kept predictions	641	Detections kept after confidence thresholding and mask-area filtering
Removed predictions	238	Low-confidence or unrealistic detections removed before severity and cost estimation

Table 8 shows that the model originally had 879 raw detections on test set however after applying the custom confidence threshold for each class, only 641 detections were kept, and 238 detections were removed because they had very small confidence scores. This indicates that prior to producing the final severity and cost outputs, the system got more selective. This filtering increases the trustworthiness of the final shown results by lowering uncertain or low-quality detections, even while it may decrease recall because some actual damage cases with low confidence ratings can be eliminated.

5.10 Severity and Cost Estimation Results

After post-processing was applied, only the accepted detections were passed to the severity estimation and repair cost estimation modules described in Chapter 4. This stage converted the model predictions into user-friendly outputs, including the detected damage type, confidence score, severity level, estimated repair cost, and overall image severity.

Table 9 shows examples of the severity and cost estimation outputs generated by the final system.

Table 9: Examples of severity and cost estimation outputs

Detected Class	Confidence	Severity	Estimated Cost (EGP)	Cost Range (EGP)	Damage Area Percentage	Overall Severity
Crack	0.808	Minor	1232	1200–2200	0.192%	Minor
Dent	0.894	Moderate	2600	600–3000	9.864%	Moderate
Dent	0.924	Moderate	2600	600–3000	8.243%	Moderate

These results proved that the system was able to take the segmentation output and convert it into cleaner assessment information for the user. For example, a small crack will be classified as minor as it takes small area of the image. While generous size dents examples will be classified as moderate or severe depending on how much area from the image it covers. If an image contains more than one damage, each damage will have its own severity and cost separately then the overall severity will be calculated based on the highest severity level detected. The cost will be considered as an approximate range, not exact quotation.

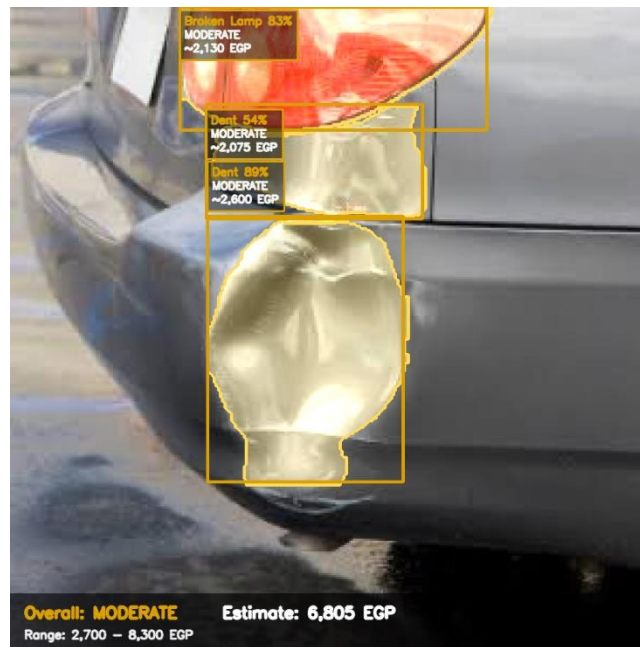


Figure 14: Example of damage detection with moderate severity, and estimated repair cost

Figure 14 shows an example output from the final system, where the detected dent is segmented, classified as moderate, and assigned an estimated repair cost of approximately 2,600 EGP within the dent repair range.

5.11 Discussion

The experimental results show that the proposed YOLOv11m-seg model achieved promising performance for vehicle damage detection and instance segmentation. The model achieved a test box mAP50 of 0.769 and a test mask mAP50 of 0.763, which indicates that it was able to detect and segment visible external vehicle damage with reasonable accuracy. The close validation and test results also suggest that the model generalized well to unseen images.

Glass shatter, tire flat, and lamp broken all performed the best since their visual characteristics are more distinct and apparent.

Dents, scratches, and cracks, on the other hand, were more difficult to detect since they are frequently tiny, inconspicuous, and visually comparable to typical car surface textures.

Because crack is typically thin and may only take up a small portion of the image, it was the weakest class.

The confusion matrix supports these findings, as most of the errors were related to missed detections of dent, scratch, and crack rather than strong confusion between damage classes. And the main issue was that the model confuses multiple classes with the background of the images because the background can have a lot of different shapes and lines that the model can confuse with scratches or cracks. However, this was reducing after post-processing because post-processing forces the model to reject low confidence predictions which can be these unrelated objects and using class-specific confidences can even help more as it can set even lower threshold for those classes

Overall, the proposed system provides a complete initial vehicle damage assessment pipeline. It detects and segments damage, filters the predictions, estimates severity, and provides an approximate repair cost. These outputs are then prepared to be displayed through the mobile application discussed in the next chapter.

5.12 Chapter Summary

This chapter presented the experimental results and discussion of the proposed YOLOv11m-seg vehicle damage detection and segmentation system. The model was trained and evaluated on the CarDD dataset using 2816 training images, 810 validation images, and 374 test images.

The final model achieved a test box mAP50 of 0.769 and a test mask mAP50 of 0.763. The results showed strong performance for glass shatter, tire flat, and lamp broken, while dent, scratch, and crack were more difficult due to their subtle appearance, small size, and similarity to vehicle surface patterns. The confusion matrix confirmed that many of the errors came from missed detections in these difficult classes.

The chapter also discussed the post-processing stage, where confidence thresholding and mask-area filtering were applied to make the outputs more suitable for user-facing results. Finally, the severity and cost estimation results showed how the detection outputs were converted into clearer information, including severity level and approximate repair cost. The next chapter presents the mobile application development, workflow, interface, and features used to deliver these results to the user.

Chapter 6: Mobile Application Development and Features

6.1 Overview of the Mobile Application

The mobile application represents the user-facing part of the proposed smart vehicle inspection and repair estimator system. The application is called FixZone and was developed to make the suggested AI model accessible to non-technical users who might not be familiar with deep learning or computer vision. The application's primary goal is to enable the user to upload or take a picture of a car, send it for damage analysis, and get a clear response that includes the type of damage found, its severity, the anticipated cost of repair, and recommendations for repair shops.

FixZone was designed as the main product of the project, it is designed to be a full system that provides vehicle damage detection, severity assessment, repair cost estimation, report production, chatbot assistance, user identification, prior scan history, repair shop recommendations, and feedback gathering. This integration was one of the main goals of the project, as the system attempts to provide a complete first assessment workflow for vehicle damage rather than only producing a detection output.

6.2 Application Development Tools and Technologies

The FixZone mobile application was developed using Flutter, which is a very known cross-platform framework that enables the creation of mobile applications with a single codebase. Android Studio was used as the main program to write and test the application on. Figma was used to design the application interface and its screens, layout, icons, and visual style before implementing

The application also used Firebase as a crucial backend service. In addition to storing user-related data, such as user information, scan history, reports, and saved results, it facilitates user

authentication. After logging onto the app, this enables each user to view their own prior scans and findings.

In addition to email and password login, the authentication system offers Google, Facebook, and X sign-up options. Additionally, the application offers basic account features like account settings, sign-out, account information modification, and password forgetting. In order to bring the app closer to a real-world mobile application experience, these features were incorporated.

6.3 System Workflow

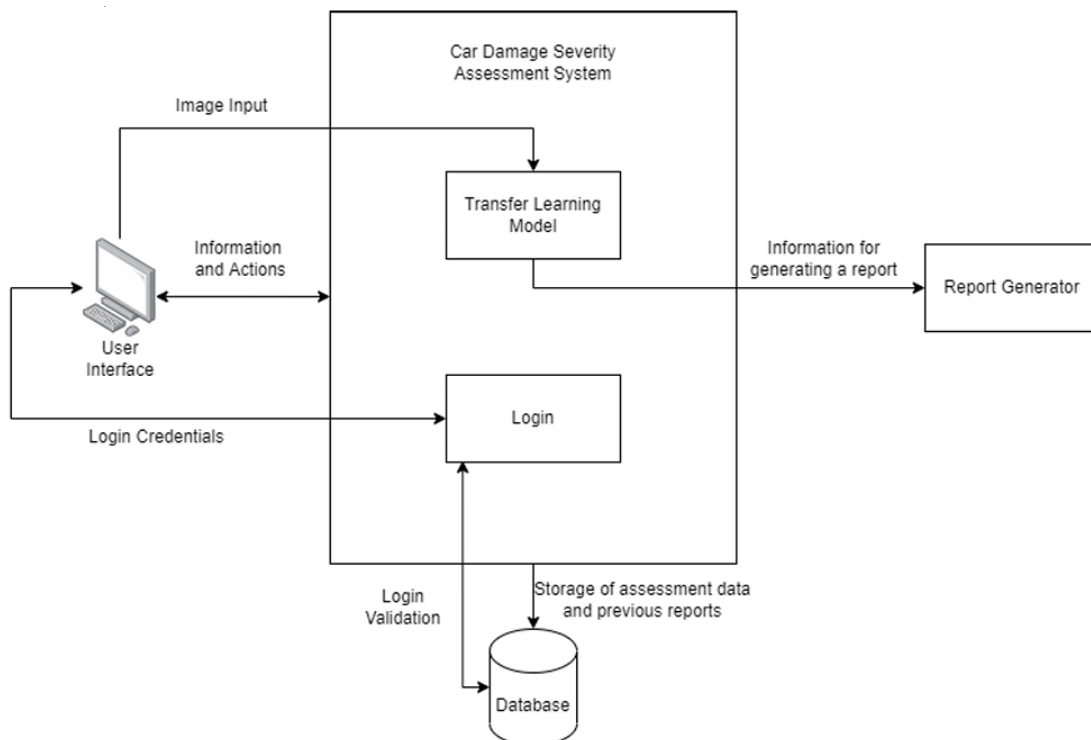


Figure 15: General workflow of the FixZone mobile application system [22]

The workflow starts when a user logs in or establishes a new account, FixZone's general workflow starts. The user is taken to the home screen after checking in, where they can upload an existing car image from the phone gallery or shoot a new picture with the camera. The application initiates the damage analysis procedure after the image is chosen.

To detect damage, the trained YOLOv11m-seg model receives the chosen image. The six supported damage categories dent, scratch, crack, glass shatter, lamp damaged, and tire flat are used by the model to identify obvious exterior vehicle damage. Even though the trained model is an instance segmentation model, the mobile application only uses bounding boxes to present the

results because it is more difficult and heavier for the mobile interface to display segmentation masks directly on the phone. Bounding boxes were therefore employed.

The severity estimation module receives the findings after detection. Based on the type of damage and the reasoning described in Chapter 4, the system determines whether each identified damage is minor, moderate, or severe. The estimated cost for each identified damage as well as the overall estimated repair cost are then determined using the repair cost estimation module. The user is then presented with the full damage analysis result by the application.

The application also provides extra services after the analysis is completed. The user can view recommended repair shops, open directions to a selected repair shop, download a PDF report, view previous scans, use the chatbot, and submit feedback.

6.4 User Interface Design

The FixZone the user interface was intended to be straightforward, contemporary, and user-friendly. The application has a constant visual identity across many screens thanks to its dark blue theme, light blue buttons, and white typography. The login screen, home screen, analysis screen, chatbot screen, account page, settings page, and feedback page all had the same design style.

The login page gives the user access to the authentication features and introduces them to the program. The primary damage assessment options, such as taking a snapshot, uploading a photo, checking prior scans, locating repair shops, and launching the chatbot, are available on the home screen. Additionally, the design has a bottom navigation bar that makes it simple for the user to switch between the account page and the home page.

The damage analysis screen displays the selected vehicle image with bounding boxes around the detected damaged areas. It also presents the scan date, overall severity, damage details, estimated cost for each damage, total estimated cost, and repair shop suggestions. This organization helps the user understand the result without needing to interpret technical model outputs.

Figure 16 shows FixZone application Login page, the home page and the damage analysis page in that order.

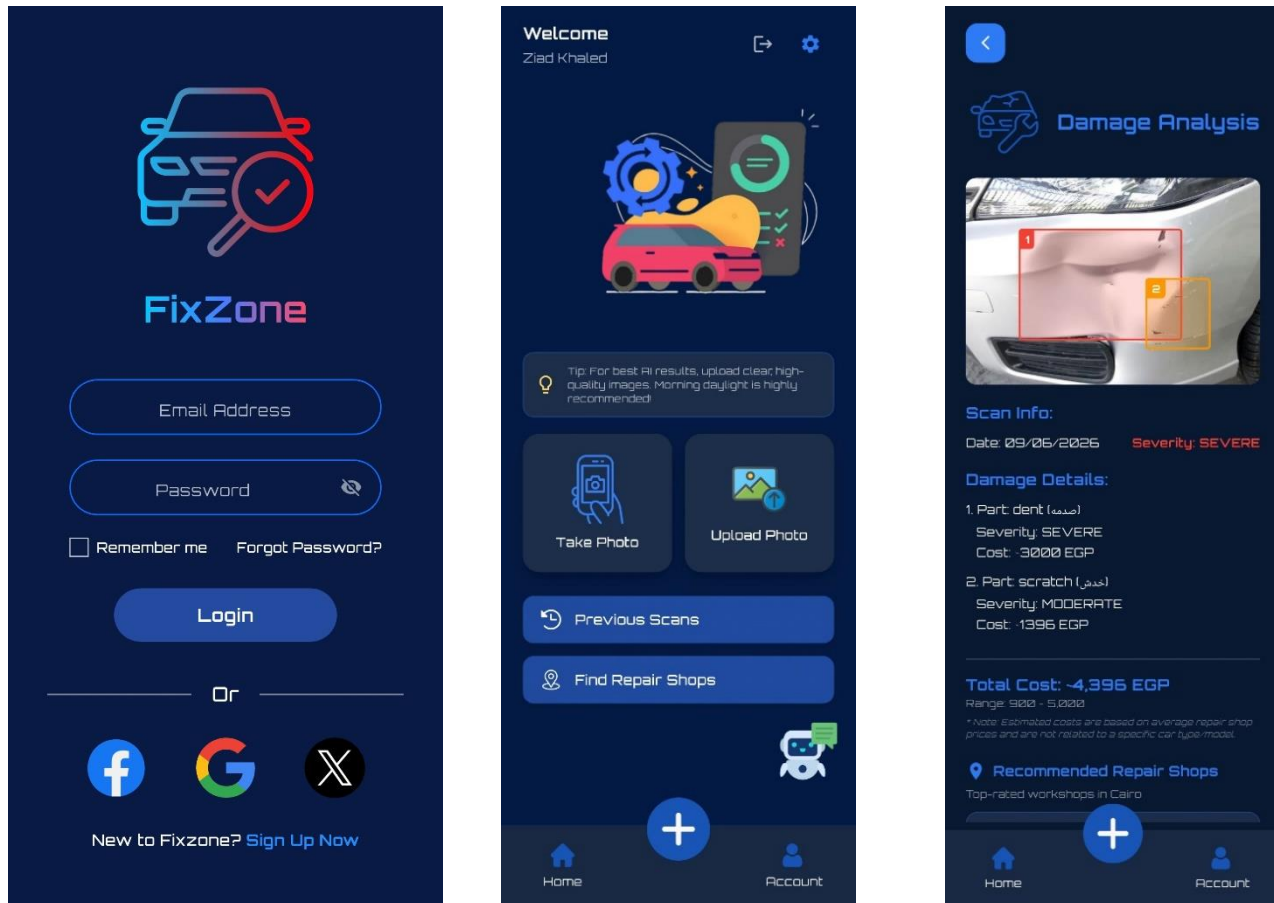


Figure 16: The login, home screen and damage analysis pages

6.5 Main Application Features

6.5.1 Login and Signup

The first part of FixZone mobile app is authentication module, which consists of two parts registration and login. Sign-up option allows new users to register for the service by entering personal information like their username, email address and password. The registration process is constructed so that it is intuitive and that all users will be able to begin using the service as soon as after they register. Registered users can log into the application and save any reports they have created of vehicle damage and any other services they receive from FixZone through their registrations.

The login and sign-up modules have great value for ensuring the security of users' personal data and creating a personalized experience for them while using the application. This will ensure the personal information and reports remain safely on the server until they are used next by other

application services. In addition, the login procedure provides a basis for the application services such as downloading reports and engaging with the bot assistant.

6.5.2 Uploading and Scanning Phase

The scan option forms part of the essential features of the FixZone mobile app, whereby users will be able to start the process of evaluating the vehicle damage. With this feature, the users can take the picture using a live image by using the camera or they can import the existing picture that is stored in the gallery of the phone.

When the image is available, the app automatically analyses the image with the help of the YOLOv11m-seg AI model that is integrated into it. The algorithm identifies and categorizes damage type in the vehicle's image including damage type: Dent, Scratch, Crack, Glass Shatter, Lamp Broken, Tire Flat. Bounding boxes are shown to mark the areas of damage and to help localize damaged parts of the vehicle. Other application attributes like damage assessment, repair cost estimation, and PDF report preparation are based on this stage.

The final mobile interface had bounding boxes shown to the user instead of segmentation masks. This decision was taken due to the fact that displaying segmentation masks on the phone interface is heavier and can make application less smooth. Hence, the bounding boxes were applied in the app to get a lighter and clearer visual result.

6.5.3 Damage Analysis Result Page

Once the analysis is done, the application shows the result of damage in the clear result page. The vehicle image is displayed with bounding boxes around the damages identified. Damage will be identified and numbered and detailed under the Damage Details section. The application displays the expected damage class, severity and repair costs for each damage.

The result page also sums up the estimated repair costs of all the damages that were detected. The overall severity of the scan is clearly shown, e.g. minor, moderate or severe. When various damages are found in the same image, multiple damages are listed in the application and severity is determined by the most severe damage found.

6.5.4 Recommended Repair Shops

After analyzing the damage, the application enhances the user convenience level by proposing relevant repair shops. The repair shop section shows a list of repair shops, their location, rating and contact number. A directions button is also available for the user to press to go to the workshop selected.

The application also allows users to filter repair shops, according to the selected location, so that the user can find the appropriate repair shop based on his location. This is the main objective of FixZone, since the application is not just for identifying the damage, but also to plan the next step that the user can take after the damage has been identified.

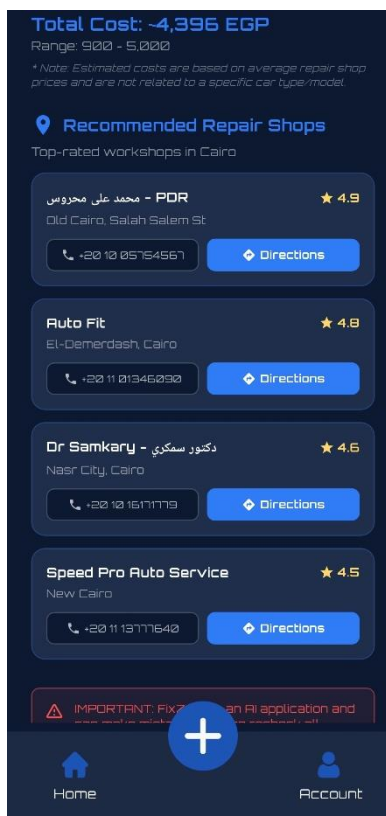


Figure 17: Repair Shops Page

6.5.5 Generated Vehicle Damage Assessment Report

After the damage detection process is over, the mobile application generates an option to download a PDF report that captures all information about the damage detection process. The application will scan the vehicle image with the help of a YOLOv11m-seg model to get the information related to any damage. The image of the vehicle will be captured with highlighted damaged areas, and any dents, scratches or other identified damage will be marked and displayed in the PDF report generated.

Each instance of damage is accompanied by a severity rating of the damage, along with a detailed estimate of the repair cost based on the type of damage. The system also determines the total damages that need to be fixed when all the damages are found.

The application also enhances the user convenience level by recommending a suitable car repair workshop that will provide the user with the opportunity to repair damages.

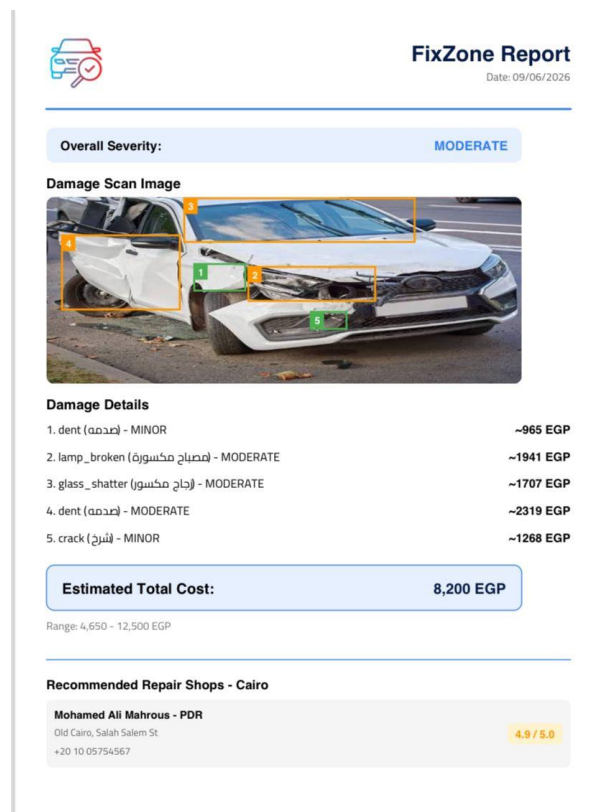


Figure 18: Generated report Layout

6.5.6 Chatbot Program

In such way, the chatbot supports the users by helping them to navigate through the app and understand the information that is provided concerning the damage analysis. It is possible to ask different questions related to the types of damages found, recommendations for repairs, information about the report itself, as well as instructions on how to use the app properly. This innovative tool makes the application accessible and easy to use for all, regardless of their level of expertise.

The chatbot has been developed as a rule-based bot. It has pre-defined responses for various questions regarding the FixZone system, damage type, damage severity, damage cost estimation, final reports, repair shops, and the general use of the app. This makes the chatbot appropriate to assisting users without needing to have an intricate generative AI system.

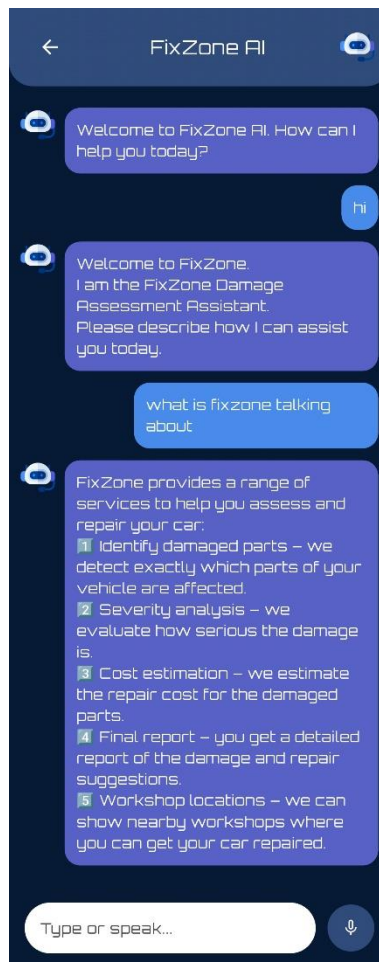


Figure 19: FixZone rule-based chatbot interface

6.5.7 Account, Settings, and User Information

The account section enables users to access and control account details. The user can view account information, previous scans, settings and sign out from the application. Account information page shows the user's information including email, first name, last name, phone number, date of birth. Some of this information can also be edited and saved within the App.

Other settings include the account page, about page, contact us page, notification control page, and feedback page. It has a notification feature to turn notifications on or off, and once the action is performed, a confirmation message is displayed.

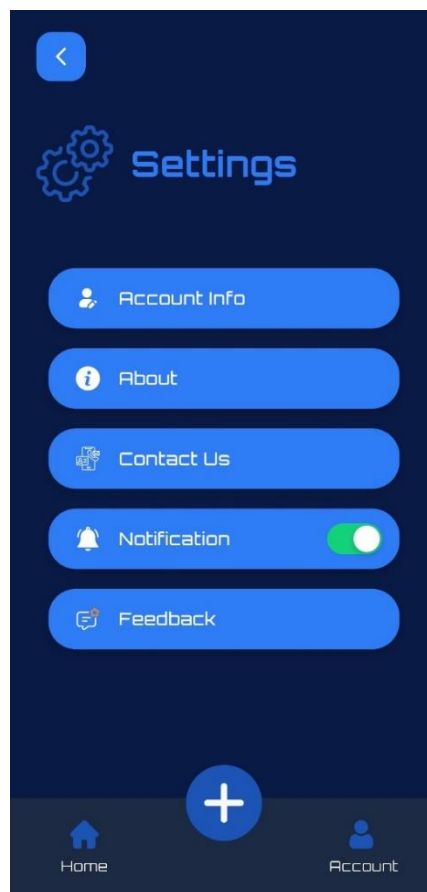


Figure 20: Settings screen of the FixZone application

6.5.8 Feedback and Rating

In addition to all of the above, FixZone contains a feedback and rating system that enables the users to evaluate the overall application experience they have once they complete the entire procedure of assessing damage to their cars. Users will be able to enter some type of feedback through a comment, where opinions and suggestions about the usability, effectiveness and accuracy of the system will be expressed. Also, the application has a 1–5-star rating system that will enable the user to assess the level of helpfulness and efficiency of the application and its services.

As mentioned previously, this is a crucial feature for the system's interaction with users and future development. Feedback and ratings gathered will provide a valuable resource to know how effective the system is and what could be improved in the future.

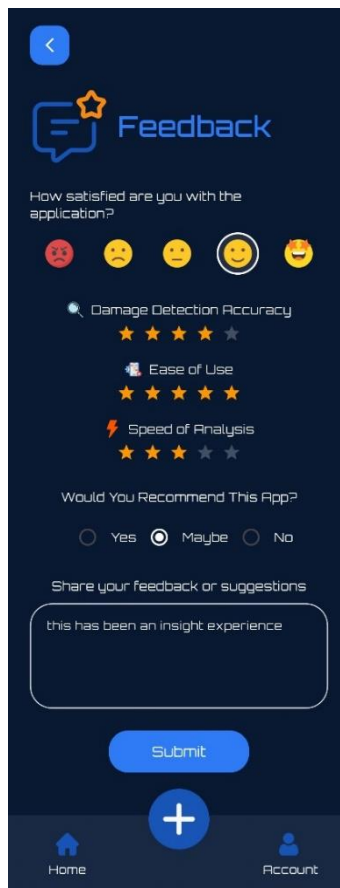


Figure 21: Feedback screen of the FixZone application

6.6 Firebase Storage and User Data Handling

The FixZone application's backend was supported by Firebase. User authentication and the storage of application-related data, such as user account information, saved scans, prior reports, and result-related data, were its primary uses. After logging into their account, each user can see their prior scan history and generated reports thanks to the application's storage of this data.

Firebase also supports the login and sign-up functionality and helps manage user credentials securely. This is important because the application includes personal account data and scan history. The use of Firebase made the application more practical because it allowed the system to connect the AI analysis results with a personalized user experience.

6.7 Advantages of the Mobile Application

The mobile application has several benefits thanks to the mobile application. First, rather than requiring users to utilize code or specialized tools, it makes the AI model accessible to them through an easy-to-use interface. Second, it offers a whole workflow that begins with uploading an image and concludes with severity, cost estimation, report creation, and recommendations for repair shops. Third, it keeps track of earlier scans and reports so that the user can access them later if necessary. Another benefit of Fixzone is that it can assist the users beyond damage detection. It provides a chatbot, recommendations for repair shops, contact details, and feedback gathering are all included in the application. The technology is more engaging and practical in real-world situations thanks to these qualities. Additionally, the application makes it very apparent that the results are estimations and should be verified by a qualified specialist.

6.8 Application Limitations

The FixZone application has certain limitations even if it offers a comprehensive first assessment methodology. The submitted image's quality determines the outcome's quality. The damage detection result may be impacted by poor illumination, indistinct photos, harsh reflections, or incorrect viewing angles. Additionally, to minimize processing and display complexity on mobile screens, the application merely shows bounding boxes rather than complete segmentation masks. The repair cost estimation is also approximate and not an exact quotation. The model of the car, the availability of spare parts, the location of the workshop, labor expenses, and market

conditions can all affect the actual cost of repairs. As a result, the program does not take the place of a professional inspection; rather, it is meant to assist the user with a first assessment.

6.9 Chapter Summary

This chapter presented the development and features of the FixZone mobile application. The application was designed using Figma and developed using Flutter through Android Studio. Firebase was used to support authentication and data storage, including user information, previous scans, and generated reports.

From login and image upload to damage analysis, severity calculation, repair cost estimation, report generation, chatbot support, repair shop recommendations, and feedback gathering, the chapter covered every step of the application's operation. All things considered, FixZone converts the suggested AI model from an experimental notebook-based system into a functional mobile application that gives customers an easily accessible initial car damage assessment tool.

Chapter 7: Conclusion and Future work

7.1 Conclusion

The current research involved the creation of a novel intelligent system for detecting vehicle damages and making repair estimates using machine learning and deep learning algorithms. The novel system leveraged the YOLOv11m-seg algorithm for carrying out the process of detection and instance segmentation on real-world vehicle images. The integration of object detection, segmentation, cost estimation, and deployment of the application on mobile devices was intended to make the process of checking cars faster, easier, and more consistent than conventional methods.

From the experiments, it was found that the proposed approach performed very well in identifying and segmenting various types of vehicle damage, like dent, scratch, crack, glass break, lamp broken, and tire flat. Performance measures like precision, recall, mean average precision at IoU of 50%, and mean average precision at IoU between 50% and 95% validated the efficiency of the YOLOv11m-seg model especially on distinct categories of vehicle damage. Confusion matrix and actual prediction outputs on vehicle images verified the ability of the model to generalize on unseen data along with localization.

Apart from that, another part of this project entailed developing an application in order to use this model and make its results available for the users through an application created via Android Studio. Users can upload pictures of their cars as well as get their assessments done automatically, including downloading a PDF document with the results, where all detected damages will be indicated, with an assessment of severity, expected cost of repairs, and possible workshops, where one can take his/her car to be repaired. Furthermore, this application includes the FixZone chatbot, which was designed to answer various questions about damage assessment and other things regarding the application itself.

In conclusion, the design of the smart vehicle inspection system is an illustration of how deep learning and computer vision techniques can enhance the efficiency and ease of the process of vehicle damage assessment. The incorporation of artificial intelligence-based vehicle damage detection and repair estimate, the generation of PDF reports, as well as the deployment of a chatbot

for users' questions and the creation of an app for smartphone devices, are features that make up a viable system for vehicle owners, insurance companies, and auto mechanics.

7.2 Limitations

FixZone still has a number of drawbacks despite offering a useful initial car damage assessment system. The quality of the uploaded image has a significant impact on the detection outcome; fuzzy photographs, dim lighting, reflections, shadows, odd angles, or partially apparent damage can all lower the accuracy of the model. Because they can be tiny, thin, or visually similar to typical car surface textures, some damage types of particularly scratches, dents, and cracks are also more challenging to identify. Furthermore, the system might not completely comprehend the precise afflicted component because the severity calculation is based on the whole and the specified rule-based logic rather than a complete car-part segmentation model. Because actual repair costs might vary based on vehicle brand, spare part availability, workshop pricing, labor costs, and market conditions, the repair cost estimate is also an estimate. As a result, the system should be viewed as an initial evaluation tool rather than a substitute for expert examination.

7.3 Future work

In the future, the system will be improved by training the model on a more extensive and varied collection of real-world vehicle damage photos, particularly those gathered from local Egyptian roads, workshops, and various car models. Additionally, more precise costs based on automobile brand, broken part, workshop location, and market updates can be added to the repair cost database. Adding car-part segmentation, which allows severity to be evaluated based on the damaged part rather than the entire image region, is another significant advance. Complete user testing, improved backend deployment, quicker analysis, and more seamless segmentation result display are other ways to enhance the mobile application.

References

- [1] “Automatic Defect Detection in Manufacturing - Vision Quality Inspection.” Accessed: Jan. 28, 2026. [Online]. Available: <https://tridentinfo.com/why-are-manual-inspection-methodsnot-effective-nowadays/>
- [2] “What are the challenges of manual inspection processes? | Simple But Needed.” Accessed: Jan. 28, 2026. [Online]. Available: <https://sbnsoftware.com/blog/what-are-the-challenges-ofmanual-inspection-processes/>
- [3] N. Manakitsa, G. S. Maraslidis, L. Moysis, and G. F. Fragulis, “A Review of Machine Learning and Deep Learning for Object Detection, Semantic Segmentation, and Human Action Recognition in Machine and Robotic Vision,” *Technologies*, vol. 12, no. 2, Jan. 2024, Accessed: Jan. 26, 2026. [Online]. Available: <https://www.mdpi.com/2227-7080/12/2/15>
- [4] A. G. Ribeiro, L. Vilaça, C. Costa, T. S. da Costa, and P. M. Carvalho, “Automatic Visual Inspection for Industrial Application,” *J. Imaging*, vol. 11, no. 10, Oct. 2025, Accessed: Jan. 26, 2026. [Online]. Available: <https://www.mdpi.com/2313-433X/11/10/350>
- [5] M. J. Hasan, C. K. Nguyen, Y. L. Boo, H. Jahani, and K.-L. Ong, “Vehicle Damage Detection Using Artificial Intelligence: A Systematic Literature Review,” *WIREs Data Min. Knowl. Discov.*, vol. 15, no. 2, p. e70027, 2025, doi: 10.1002/widm.70027.
- [6] M. R. S. Ramazhan, A. Bustamam, and R. A. Buyung, “Smart Car Damage Assessment Using Enhanced YOLO Algorithm and Image Processing Techniques,” *Information*, vol. 16, no. 3, p. 211, Mar. 2025, doi: 10.3390/info16030211.
- [7] V. Ghita, D. Iorga, L.-M. Neagu, M. Dascalu, and G. Militaru, “AI for Car Damage Detection and Repair Price Estimation in Insurance: Market Research and Novel Solution,” in *Rethinking Business for Sustainable Leadership in a VUCA World*, M. Busu, Ed., in Springer Proceedings in Business and Economics. , Cham: Springer Nature Switzerland, 2024, pp. 167–179. doi: 10.1007/978-3-031-50208-8_10.
- [8] M. Dwivedi *et al.*, “Deep Learning-Based Car Damage Classification and Detection,” in *Advances in Artificial Intelligence and Data Engineering*, vol. 1133, N. N. Chiplunkar and T. Fukao, Eds., in *Advances in Intelligent Systems and Computing*, vol. 1133. , Singapore:

Springer Nature Singapore, 2021, pp. 207–221. doi: 10.1007/978-981-15-3514-7_18.

- [9] “Deep Learning Based Car Damage Detection, Classification and Severity,” *Int. J. Adv. Trends Comput. Sci. Eng.*, vol. 10, no. 5, pp. 2947–2953, Oct. 2021, doi: 10.30534/ijatcse/2021/031052021.
- [10] R. E. Van Ruitenbeek and S. Bhulai, “Convolutional Neural Networks for vehicle damage detection,” *Mach. Learn. Appl.*, vol. 9, p. 100332, Sep. 2022, doi: 10.1016/j.mlwa.2022.100332.
- [11] V. Mistry, C. Jagad, U. Jhunjhunwala, and P. Tawde, “Intelligent Car Damage Detection System,” in *Innovations in VLSI, Signal Processing and Computational Technologies*, vol. 1095, G. Mehta, N. Wickramasinghe, and D. Kakkar, Eds., in Lecture Notes in Electrical Engineering, vol. 1095, Singapore: Springer Nature Singapore, 2024, pp. 477–484. doi: 10.1007/978-981-99-7077-3_47.
- [12] P. Warungse, A. Kasar, A. Kshirsagar, G. Hade, and N. Khandhar, “Car Damage Detection Using Computer Vision,” vol. 10, no. 11, 2023.
- [13] S. A. Pérez-Zarate, D. Corzo-García, J. L. Pro-Martín, J. A. Álvarez-García, M. A. Martínez-del-Amor, and D. Fernández-Cabrera, “Automated Car Damage Assessment Using Computer Vision: Insurance Company Use Case,” *Appl. Sci.*, vol. 14, no. 20, p. 9560, Oct. 2024, doi: 10.3390/app14209560.
- [14] M. Vijay, V. Vijay, L. Jegan, and B. Rama, “Car Damage Detection Using Machine Learning,” *Int. J. Creat. Res. Thoughts IJCRT*, vol. 12, no. 5, 2024.
- [15] “(PDF) Enhancing Robustness And Generalization In Vehicle Damage Detection Through Deep Neural Networks,” *ResearchGate*, Aug. 2025, Accessed: Jan. 28, 2026.
[Online]. Available:
https://www.researchgate.net/publication/382968236_Enhancing_Robustness_And_Generalization_In_Vehicle_Damage_Detection_Through_Deep_Neural_Networks
- [16] Dr.G. Uma Maheswari, Jabez Jerin J, John Rathinam V, and Sunil P, “CAR Damage Price Predictor,” *Int. Res. J. Adv. Eng. Hub IRJAEH*, vol. 3, no. 03, pp. 522–526, Mar. 2025, doi:

10.47392/IRJAEH.2025.0073.

- [17] D. Z. Baig, M. Kamal, and Z. Ullah, “Small Dents, Big Impact: A Dataset and Deep Learning Approach for Vehicle Dent Detection,” Sep. 26, 2025, *arXiv*: arXiv:2508.15431. doi: 10.48550/arXiv.2508.15431.
 - [18] S. Kang, Z. Hu, L. Liu, K. Zhang, and Z. Cao, “Object Detection YOLO Algorithms and Their Industrial Applications: Overview and Comparative Analysis,” *Electronics*, vol. 14, no. 6, Mar. 2025, Accessed: Jan. 26, 2026. [Online]. Available: <https://www.mdpi.com/2079-9292/14/6/1104>
 - [19] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” May 09, 2016, *arXiv*: arXiv:1506.02640. doi: 10.48550/arXiv.1506.02640.
 - [20] R. Sapkota *et al.*, “YOLO advances to its genesis: a decadal and comprehensive review of the You Only Look Once (YOLO) series,” *Artif. Intell. Rev.*, vol. 58, no. 9, p. 274, Jun. 2025, doi: 10.1007/s10462-025-11253-3.
 - [21] X. Wang, W. Li, and Z. Wu, “CarDD: A New Dataset for Vision-based Car Damage Detection,” *IEEE Trans. Intell. Transport. Syst.*, vol. 24, no. 7, pp. 7202–7214, Jul. 2023, doi: 10.1109/TITS.2023.3258480.
 - [22] A. T. Mohamad, N. A. S. Abdullah, S. Ahmad, and G. B. Francis, “Car Damage Severity Assessment Using Supervised Deep Learning,” Dec. 2024. Accessed: Jun. 13, 2026. [Online]. Available: <https://www.atlantis-pess.com/proceedings/invent-24/126005601>
- Link to the survey conducted: <https://forms.gle/TULAA5RfSSGeCYdA6>